

Attention-Based Sequence Models for Security Patch Detection Using Behavioural Data

William Godfrey

Supervisor: Dr Neetesh Saxena

Moderator: Dr Oktay Karakus

BSc Computer Science
School of Computer Science and Informatics
Cardiff University

5th May 2026

Abstract

Silent security patches, fixes committed to public repositories before formal disclosure, create a window in which attackers monitoring commit feeds can reverse-engineer vulnerabilities ahead of affected users. Prior detectors operate on commit messages or source code, both of which fail when developers omit vulnerability references. Behavioural detection classifies commits from repository event metadata alone, but existing work uses only convolutional architectures, evaluates with future-aware windows unavailable at commit time, and shares repositories across train and test folds.

This dissertation evaluates a pre-norm transformer encoder against a Conv1D baseline on behavioural repository data under a repository-grouped, multi-seed protocol. Eighty training runs span four window sizes ($B = 5, 10, 20, 40$), symmetric and causal modes, five seeds, ten GroupKFold folds, supporting paired t -tests, Wilcoxon, bootstrap CIs, and Cohen’s d . External validation uses 212 CVEfixes repositories; 29 are truly held out (no name match or commit-hash overlap with training).

The transformer reaches 87.0% held-out accuracy at $B = 20$ symmetric (seed 42; cross-seed 0.847 ± 0.018), improving over Conv1D significantly (accuracy $p = 0.002$, $d = 1.36$; F1 $p < 0.001$, $d = 1.68$). Causal accuracy (0.849–0.852) matches symmetric, while Conv1D collapses to 0.694 at causal $B = 5$, making commit-time detection feasible. At seed 42, symmetric $B = 5$, the transformer reaches 87.6% accuracy / 95.1% recall on the near-domain set and 86.8% / 97.4% on the truly-unseen subset. On causal $B = 20$ the same model reaches **90.7% accuracy, 91.4% F1, 97.4% recall** on the truly-unseen subset, exceeding its near-domain accuracy (88.1%) by 2.6 points, without ever reading repository name, commit message, or source code. Ablation shows the twenty-three new temporal features contribute more than the architectural change; eliminated repository-level leakage accounted for approximately six points of inflated accuracy.

Contents

1	Introduction	8
1.1	Motivation and Context	8
1.2	Research Problem	8
1.3	Aim and Objectives	9
1.4	Challenges	9
1.5	Project Risks	10
1.6	Contributions	10
2	Background and Related Work	11
2.1	Overview	11
2.2	Sequence Modelling and Attention Mechanisms	11
2.3	Software Vulnerabilities and Patch Disclosure Dynamics	11
2.4	Security Patch Detection Approaches	12
2.5	Rationale for Transformer-Based Behavioural Detection	13
2.6	Dataset Construction and Evaluation Constraints	14
2.7	Commercial and Non-Commercial Tools	14
3	Methodology	15
3.1	Overview	15
3.2	Research Approach	15
3.3	Research Design	15
3.4	Methodological Corrections	16
3.5	Justification of Evaluation Protocol	17
4	Design and Implementation	18
4.1	Overview	18
4.2	System Model	18
4.3	Threat Model	22
4.4	Dataset Details	22
4.4.1	Training Data Source and Scope	22
4.4.2	External Validation Data	23
4.5	Experimental Setup	24
4.6	Experimental Design	24
4.6.1	Event Window Extraction	24

4.6.2	Feature Engineering	25
4.6.3	Model Architectures	26
4.6.4	Training Procedure	29
4.6.5	Evaluation Metrics	29
5	Results and Evaluation	30
5.1	Overview	30
5.2	Results	30
5.2.1	Transformer Results	30
5.2.2	Conv1D Results	30
5.2.3	External Validation Results	35
5.3	Evaluation	36
5.3.1	Transformer versus Conv1D: Window Size Effect	36
5.3.2	Impact of Removing Future Context	36
5.3.3	Window Size Consolidation	37
5.3.4	Statistical Significance	37
5.3.5	Feature Importance	38
5.3.6	Architecture Exploration	40
5.3.7	External Validation: Seed-42 Model on Enriched CVEfixes	40
5.3.8	Comparison to Prior Work	42
5.3.9	Error Analysis	43
5.3.10	Attention Pattern Analysis	44
5.3.11	Summary of Findings and Limitations	45
6	Conclusion and Future Work	46
6.1	Conclusions	46
6.2	Future Work	47
7	Reflection on Learning	48
7.1	Evaluation Rigour versus Inherited Methodology	48
7.2	Debugging Inherited Code	48
7.3	Feature Engineering versus Architecture Complexity	48
7.4	Data Alignment and External Validation	49
7.5	Time Management and Experimental Planning	49
7.6	Reproducibility as Engineering Discipline	49
7.7	Use of AI Assistance	49
	References	51

List of Figures

4.1	Contribution map for the overall system. Components inherited from Farhi et al. (2023) are shown with dashed grey borders; components added or rewritten for this project are shown with solid borders. Full component list in Table 4.1.	19
4.2	Transformer training and evaluation pipeline. Preprocessed event windows of shape $(n, T, \sim 435)$, where the time dimension T depends on window size B and pooling mode (symmetric or causal, $B \in \{5, 10, 20, 40\}$), enter a 5-seed, 10-fold GroupKFold loop grouped by repository. Per-fold transformer models are trained with AdamW and a cosine-warmup schedule, then averaged as an ensemble. Held-out metrics, CVEfixes external validation, statistical tests, and attention and error analysis are computed from the same ensemble predictions.	21
4.3	Symmetric versus causal window extraction. Symmetric windows include events on both sides of the target; causal windows include only events strictly before the target with the target as the final timestep.	25
4.4	Full transformer classifier used in this project: 2 pre-norm encoder blocks (4 heads, $d_{\text{model}} = 64$, FFN $64 \rightarrow 256 \rightarrow 64$), window-aware pooling, and a 128/64 ReLU sigmoid head ($\sim 147\text{k}$ parameters).	28
5.1	Window size comparison showing ensemble accuracy and F1 for transformer and Conv1D across $B = 5, 10, 20, 40$ (symmetric).	31
5.2	Confusion matrices comparing transformer and Conv1D at $B = 20$ (symmetric, seed 42).	32
5.3	Transformer confusion matrices under causal windows across $B = 5, 10, 20, 40$ (seed 42). Accuracy holds at 88.0% even at $B = 5$	32
5.4	Conv1D confusion matrices under causal windows across $B = 5, 10, 20, 40$ (seed 42). The model collapses at $B = 5$ (66.1% accuracy) and recovers at larger windows.	33
5.5	Multi-seed robustness showing ensemble accuracy and F1 across 5 seeds.	33
5.6	Grid confusion matrices for the best configuration ($B = 20$, symmetric, transformer) across all 5 seeds.	34
5.7	Averaged confusion matrix for the best configuration across 5 seeds (84.7% overall accuracy).	35
5.8	Per-fold performance comparison across 10 GroupKFold folds.	38
5.9	Paired differences (Transformer minus Conv1D) per fold as box plots.	38

5.10	Permutation importance of top 30 features. Blue bars indicate new features; grey bars indicate original.	39
5.11	ROC and precision-recall curves for the ensemble (seed 42, $B = 20$ symmetric, held-out test set). AUC = 0.919, AP = 0.891.	44
5.12	Per-head attention patterns from Layer 1 ($B = 20$ symmetric, seed 42). Top row: averaged attention weights for vulnerable samples on a sequential “hot” colormap, where deeper red indicates a higher attention weight from the query position (rows) onto the key position (columns). Bottom row: the difference map (vulnerable – benign) on a diverging red–blue colormap, where red cells mark query–key pairs that receive more attention on vulnerable samples than on benign ones, blue cells mark pairs that receive less, and white cells indicate no class-conditional difference; each colour bar reports the per-head numerical scale.	45

List of Tables

2.1	Attack categories associated with patch disclosure.	12
2.2	Representative security patch detection approaches.	13
2.3	Industry vulnerability management tools.	14
3.1	Eight most consequential methodological corrections.	16
4.1	Component inventory: inherited versus newly contributed components across the four functional layers.	20
4.2	Threat risk matrix.	22
4.3	Feature categories and counts per timestep.	26
4.4	Structural comparison between the transformer encoder and the Conv1D baseline.	27
5.1	Ensemble performance under symmetric windows (before_cve, centre pooling). Mean $\pm$ std across 5 seeds.	30
5.2	Ensemble performance under causal windows (only_before, last pooling). Mean $\pm$ std across 5 seeds.	31
5.3	Cross-seed robustness ($B = 20$, symmetric, transformer, ensemble on held-out test set).	31
5.4	CVEfixes external validation per seed (transformer, $B = 5$, symmetric, enriched). Top: near-domain (212 repos including 183 duplicates of training repos at distinct commits, detected as case-insensitive name matches plus an owner-rename caught by hash overlap). Bottom: truly-unseen (29 repos with no name match and no commit-hash overlap against the training set).	36
5.5	Transformer advantage over Conv1D (delta in ensemble accuracy, percentage points). 5-seed averages.	37
5.6	Statistical significance: Transformer vs Conv1D ($B = 20$, symmetric, seed 42, 10-fold GroupKFold).	37
5.7	Feature ablation results ($B = 20$, symmetric, transformer, seed 42).	39
5.8	Top 10 features by permutation importance (ensemble of 10 fold models, $B = 20$, seed 42).	39
5.9	Key architecture and training experiments (seed 42, symmetric).	40

5.10	Near-domain CVEfixes external validation: seed-42 trained models, one per configuration, evaluated on the full 212-repository CVEfixes set (2,403 balanced samples). Includes 183 duplicates of training repositories (case-insensitive name matches plus one owner-rename caught by commit-hash overlap) sampled at unseen commits.	41
5.11	Truly-unseen window sweep: same 16 seed-42 trained models evaluated on the 29 CVEfixes repositories with no name match (case-insensitive) and no commit-hash overlap against the training set (152 balanced samples).	41
5.12	Result comparisons with previous security patch detection studies.	42

Chapter 1

Introduction

1.1 Motivation and Context

As technology continues to evolve, open-source software and collaborative development platforms have become essential to modern software engineering. Version control systems such as Git allow developers to collaborate, store and release updates, but security risks can be introduced through unpatched software and poor communication regarding vulnerabilities. In recent years this problem has been growing substantially, with over 21,500 new CVEs disclosed within just the first six months of 2025, an 18% increase from the same period the previous year, and over a third rated high or critical (Khalil 2025). These numbers highlight how important it is for software to be kept secure and up to date.

Security patches should be announced within a reasonable time frame alongside formal disclosure into databases such as the Common Vulnerabilities and Exposures (CVE) programme. However, some developers patch silently, leaving users unaware and vulnerable. The 2021 Log4Shell vulnerability (CVE-2021-44228) affected hundreds of millions of devices, and early patches were committed to the Apache repository days before a formal advisory was issued, leaving most users exposed during the gap (Hiesgen et al. 2024). In 2024, a backdoor was discovered in the xz compression library (CVE-2024-3094) that had been deliberately introduced through years of seemingly benign contributions, only caught by accident before it reached most production systems (Goodin 2024). Even major vendors are not immune: a 2010 Microsoft update silently patched three vulnerabilities without any disclosure to customers (Reuters 2010). In each case, the time between when a fix became publicly visible in the codebase and when users were actually notified ranged from days to weeks, creating a window where attackers who monitor public repositories can reverse-engineer patches and exploit unpatched systems.

1.2 Research Problem

Earlier work on automated security patch detection has focused on analysing source code changes and commit messages. While these approaches can be effective, they are programming-language specific and rely on developers accurately describing their changes, which is precisely the assumption that fails when patches are applied silently. Behavioural approaches (Farhi

et al. 2023) sidestep this by classifying commits from the patterns of repository activity like forks, issues and releases surrounding each commit, rather than the content of code or messages.

Despite promising results, a few issues remain. The event window construction used by Farhi et al. (2023) includes future context, events that happen after the target commit, which would not be available in a real deployment scenario where predictions need to be made at commit time. Farhi et al. (2023) explicitly identifies window configuration as a key parameter but does not evaluate it in depth, and there is limited understanding of how behavioural models perform when restricted to only pre-commit data. Furthermore, transformers have been shown to outperform other architectures on commit-message text (Zuo et al. 2023) and Java patches (Stefanoni et al. 2022) but have never been applied to purely behavioural repository data.

1.3 Aim and Objectives

The aim is to evaluate whether a transformer-based sequence model applied to behavioural repository data can improve security patch detection under realistic temporal constraints. This aim is decomposed into three research questions:

- **RQ1:** Does a transformer encoder achieve statistically significant improvements over the Conv1D baseline (Farhi et al. 2023) on behavioural repository data?
- **RQ2:** Does removing future context from the event window substantially degrade detection performance?
- **RQ3:** How does event window size influence detection accuracy, and does the transformer’s advantage vary with window size?

Both models are evaluated across four window sizes ($B = 5, 10, 20, 40$) and two temporal positioning modes (symmetric and causal), with five random seeds (42, 123, 7, 256, 999) and GroupKFold cross-validation preventing data leakage. External validation is performed on 212 repositories from the CVEfixes database (Bhandari et al. 2021), of which 29 have no name match (case-insensitive) and no commit-hash overlap with the training set; both views are reported.

1.4 Challenges

Three challenges shaped the project. First, adapting an inherited codebase required identifying and correcting methodological flaws without destabilising the training pipeline. Second, constructing an independent external validation set from the CVEfixes database required reconciling differences in event type coverage between sources. Third, running 80 training configurations on a single consumer GPU (RTX 3060, 6 GB VRAM) required careful memory management and experimental scheduling.

1.5 Project Risks

The main risk was that behavioural data alone would lack enough signal to discriminate silent patches, in which case the transformer would not improve over the baseline. This was mitigated by planning feature engineering in parallel with architectural experiments. A second risk was that external validation could fail due to distributional mismatch; this was mitigated by designing the scraping pipeline to match the training event schema. A third risk concerned reproducibility: a previously uncontrolled source of variance (non-deterministic file ordering) was discovered mid-project and had to be eliminated retroactively.

1.6 Contributions

This project makes three contributions:

1. **A transformer-based detector for behavioural repository data.** The model is a pre-norm encoder stack with learned positional embeddings, padding-mask handling, window-aware pooling (center-token for symmetric windows, last-token for causal), and a three-layer classification head, trained with AdamW and a cosine-warmup schedule. No prior behavioural patch detection work uses attention; Farhi et al. (2023) use Conv1D and LSTM only. The transformer delivers statistically significant gains over the Conv1D baseline (accuracy $p = 0.002$, $d = 1.36$; F1 $p < 0.001$, $d = 1.68$).
2. **Systematic evaluation under causal (pre-commit-only) windows.** No prior study evaluates behavioural detection without future context. The transformer maintains its accuracy under causal constraints while the Conv1D baseline collapses to near-random at short windows, establishing whether this family of models could be deployed at commit time.
3. **Twenty-three new behavioural features and a rigorous 5-seed GroupKFold protocol.** The new features account for the top-9 slots in permutation importance and contribute more to the final accuracy than the architecture change. The stricter evaluation protocol exposed approximately 6 points of inflated accuracy in the inherited methodology. The deployment-relevant causal $B = 20$ configuration, evaluated on 29 truly-unseen repositories (no name match, no commit-hash overlap against training), achieves **90.7% accuracy, 91.4% F1, and 97.4% recall**, exceeding the same configuration’s near-domain accuracy of 88.1% by 2.6 points despite the model never seeing repository name, commit message, or source code, and demonstrating that a behavioural detector can generalise to repositories never observed during training.

The scope is intentionally limited to publicly available data. The task is binary classification distinguishing security-relevant commits from benign ones. The project does not aim to develop a production-ready tool. As no sensitive data is utilised, formal ethical approval is not required.

Chapter 2

Background and Related Work

2.1 Overview

In this chapter, I will detail the theoretical foundations of sequence modelling and attention mechanisms, the vulnerability disclosure dynamics that motivate automated detection, the existing approaches for security patch detection, the rationale for applying transformers to behavioural data, the evaluation constraints that affect reported performance, and the commercial tools that operate alongside academic systems.

2.2 Sequence Modelling and Attention Mechanisms

Repository behavioural data can be represented as temporally ordered event sequences. Modelling such data requires architectures capable of capturing relationships between non-adjacent events. Prior behavioural security patch detection approaches have predominantly employed recurrent architectures (Farhi et al. 2023), which process inputs sequentially and propagate information through hidden states.

Transformer architectures replace recurrence with self-attention, allowing each position in a sequence to directly attend to all other positions (Vaswani et al. 2017). This enables modelling of global contextual dependencies without reliance on stepwise state propagation. In behavioural repository data, security-relevant patterns may be temporally dispersed rather than adjacent to the target commit. For example, a burst of issue activity days before a commit, or a change in push frequency weeks prior, could both carry useful signal.

Transformer models have demonstrated strong performance in textual security patch classification (Zuo et al. 2023), yet their application to purely behavioural repository data remains untested. Behavioural sequences differ from text in important ways: they are shorter, noisier, and lack the grammatical structure that language models exploit. Whether transformers still provide benefit in this setting is an open question.

2.3 Software Vulnerabilities and Patch Disclosure Dynamics

Software vulnerabilities are weaknesses that can be exploited to compromise confidentiality, integrity, or availability. Industry reports (Verizon 2025; Khalil 2025) indicate that vulner-

ability exploitation represents a significant and increasing percentage of security breaches, reinforcing the need for timely disclosure.

The management of vulnerabilities ideally follows a disclosure-patch-deployment lifecycle. However, these steps are not mandatory, and many developers fail to follow them, whether due to time pressure or reputational damage. Security patches are often published before formal vulnerability disclosure, and every patch remains publicly inspectable. Table 2.1 summarises the primary threats associated with patch disclosure.

Table 2.1: Attack categories associated with patch disclosure.

Threat Category	Mechanism	Temporal Characteristics	Adversarial Advantage	Implications
Patch-gap exploitation	A fix is committed but systems have not deployed it	Between patch availability and widespread deployment	Exploitation of known-vulnerable systems	Faster identification narrows the exploitable window
Patch-diff reverse engineering	Attackers analyse public diffs to infer the vulnerability	Immediately after a patch is committed or merged	Accelerated exploit development	Rapid detection limits informational asymmetry
Rapid n-day exploitation	Exploits developed shortly after disclosure	Within days of disclosure	Compresses defender response time	Detection must operate close to commit time
Silent patch exposure	Security commits not explicitly labelled	Before co-ordinated disclosure	Attackers identify changes first	Behavioural inference reduces reliance on labels
Supply-chain propagation	Vulnerabilities inherited by dependent projects	Persists until downstream updates	Amplifies impact beyond original repository	Upstream identification supports dependency management
Incomplete disclosure metadata	CVE records lack linkage to specific commits	During co-ordination processes	Delays defensive mapping	Repository-level methods reduce centralised dependence

2.4 Security Patch Detection Approaches

Existing approaches can broadly be grouped into three methodological categories: commit-message based, code-diff based, and behavioural.

Early approaches primarily used textual signals from commit messages, combining LSTM and CNN layers over code revisions (Zhou et al. 2021) and more recently transformer architectures applied to textual inputs (Zuo et al. 2023). However, silent security patches deliberately omit explicit references to vulnerabilities (Zhou et al. 2021), making linguistic detection ineffective in exactly the scenario that motivates this field.

Subsequent research combined code revision analysis with textual features. Zhou et al.

(2021) design a system combining commit-message and code-revision neural networks on C projects, while Stefanoni et al. (2022) benchmark deep learning models for Java. This language specificity limits how well these approaches generalise.

Multi-modal approaches fuse several signal types. PatchView (Farhi et al. 2025) achieves 94.5% accuracy by combining behavioural, textual, and code-based sub-models using RoBERTa and CodeBERT alongside a Conv1D behavioural encoder. VulCurator (Nguyen et al. 2022) combines commit messages, issues, and code with transformer-based language models, achieving 89.0% F1.

Behavioural approaches take a different direction. Farhi et al. (2023) identify security patches using only behavioural signals derived from GitHub repository activity, achieving 83.9% accuracy with a Conv1D model. This is language-agnostic by design and does not depend on developers describing their changes accurately. However, the Conv1D kernel of size 2 limits relationships to adjacent event pairs, and the evaluation includes post-commit events. Table 2.2 summarises representative studies.

Table 2.2: Representative security patch detection approaches.

Study	Input Representation	Methodological Characteristics	Reported Performance
Zhou et al. (2021)	Commit messages + code revisions (C)	Hybrid LSTM + CNN, language-specific	F1 $\sim$ 80–92%
Stefanoni et al. (2022)	Java patches, tokenised	LSTM and Transformer on Java	F1 $\sim$ 87%
Zuo et al. (2023)	Commit messages only	Transformer, language-agnostic text	Acc $\sim$ 91%, F1 $\sim$ 89.1%
Wang et al. (2021)	Messages + code (C/C++)	Twin RNN with bi-LSTM	Acc $\sim$ 83.6%, F1 $\sim$ 74.7%
Farhi et al. (2023)	Behavioural GitHub events	LSTM/GRU/Conv1D, language-oblivious	Acc $\sim$ 83%, F1 $\sim$ 84.1%

2.5 Rationale for Transformer-Based Behavioural Detection

In this section I set out why the combination of behavioural data and a transformer encoder has not been evaluated before and why it is a reasonable choice.

The most important reason is that behavioural data is the only input type that works regardless of programming language and regardless of whether the developer discloses the security relevance of the commit. Text-based methods fail when developers do not describe their fixes: Zhou et al. (2021) report that keyword filtering over four large C projects retains only 7.4% of commits as potentially security-relevant, and many remain silent. Code-based methods fail when applied to languages they were not designed for. Behavioural signals are available for any repository on any platform.

The second reason is architectural. The Conv1D model used by Farhi et al. (2023) has a kernel size of 2 and can only relate pairs of adjacent events. Security-relevant patterns are unlikely to be limited to consecutive events. Self-attention relates events at any pair of positions in a single operation, which is a natural fit for this type of data.

The third reason is temporal realism. Prior behavioural evaluations include post-commit events, but in practice predictions need to be made at commit time. The transformer’s ability to attend to any position in the sequence should help when the window is restricted and adjacent events happen to be uninformative.

2.6 Dataset Construction and Evaluation Constraints

The validity of reported performance is strongly influenced by how datasets are constructed and evaluated. Many studies build datasets retrospectively by linking commits to disclosed CVE records (Zhou et al. 2021; Wang et al. 2021), which introduces disclosure bias. Evaluation methodology also matters: random train-test splits over aggregated datasets (Wang et al. 2021; Stefanoni et al. 2022) may allow commits from the same repository in both training and testing. Farhi et al. (2023) partitions by repository but does not use explicit GroupKFold or report per-fold variance. Event-window construction that includes post-commit events (Farhi et al. 2023) further inflates results relative to deployment conditions.

2.7 Commercial and Non-Commercial Tools

Industry systems are disclosure-driven and signature-based rather than predictive. The CVE programme and the National Vulnerability Database catalogue publicly disclosed vulnerabilities. GitHub Security Advisories and Dependabot trigger alerts when dependency versions match known records. Software Composition Analysis platforms such as Snyk and Black Duck extend this by cross-referencing dependencies. Static Application Security Testing tools such as Checkmarx analyse source code for predefined patterns. Table 2.3 compares these tools.

Table 2.3: Industry vulnerability management tools.

System / Tool	Detection Mechanism	Structural Limitations
CVE / NVD Infrastructure	Centralised database of disclosed vulnerabilities	Entirely disclosure-dependent; retrospective
GitHub Advisories and Dependabot	Alerts when dependencies match known records	Version-matching only; does not classify commits
Software Composition Analysis	Cross-references component versions with databases	Bounded by database coverage; misses silent patches
Static Application Security Testing	Rule-based code analysis for weakness patterns	Detects coding flaws, not commit classification

In summary, behavioural data uniquely addresses silent-patch scenarios, transformer architectures offer structural advantages over local convolutions for dispersed temporal patterns, evaluation protocols substantially influence reported performance, and industry tools remain disclosure-driven rather than predictive.

Chapter 3

Methodology

3.1 Overview

In this chapter, I will describe the research methodology: the qualitative and quantitative approaches taken, the research design, the methodological corrections applied to the inherited codebase, and the justification for the chosen evaluation protocol.

3.2 Research Approach

The project combines quantitative and qualitative components. The quantitative component is an empirical comparison of two architectures (transformer and Conv1D) across four window sizes, two temporal modes, and five random seeds, producing 80 training configurations. Quantitative evaluation uses accuracy, precision, recall, F1, and ROC-AUC, with paired t-tests, Wilcoxon signed-rank tests, and bootstrap confidence intervals to establish significance. The qualitative component consists of permutation-importance inspection, architecture exploration across 23 variants, and attention-pattern analysis to interpret what the model learns.

3.3 Research Design

The research design is comparative and controlled. Each configuration is evaluated under identical data splits and preprocessing, with GroupKFold cross-validation ($k = 10$) grouping samples by source repository to prevent data leakage. External validation is performed on 212 CVEfixes repositories (Bhandari et al. 2021); of these, 29 satisfy both a case-insensitive name-match check and a 1% commit-hash overlap check against the training set, forming the truly-unseen subset. Both the full near-domain set and the truly-unseen subset are reported in Chapter 5 so the contribution of cross-commit generalisation can be separated from cross-repository generalisation. Multi-seed replication (5 seeds) quantifies the sensitivity of results to benign commit selection and model initialisation.

3.4 Methodological Corrections

This project builds upon the codebase of Farhi et al. (2023). During adaptation, nineteen issues were identified and corrected. The most consequential was replacing standard k-fold with GroupKFold, which reduced reported accuracy by approximately 6 percentage points by eliminating data leakage. Other critical corrections included a list mutation error causing silent sample loss, the early stopping callback not being passed to `model.fit()`, the causal window excluding the target commit, division-by-zero in normalisation, and non-deterministic file ordering causing 2–3% variance. Table 3.1 lists the eight most consequential corrections.

Table 3.1: Eight most consequential methodological corrections.

Issue	Impact	Correction
Standard k-fold cross-validation	Data leakage	Replaced with GroupKFold, grouping by repository index
List mutation during iteration	Silent, variable sample loss	Constructed new list instead of modifying in place
Early stopping callback not passed	Always ran full epochs; over-fitting	Included callback in callbacks list passed to <code>model.fit()</code>
Causal window excluded target commit	Classification signal removed	Adjusted slice boundary from $[i - B : i]$ to $[i - B : i + 1]$
Division by zero in normalisation	NaN propagation	Zero-filled column when variance = 0 (original divided unguarded)
Non-deterministic file ordering	2–3% variance between identical runs	Applied <code>sorted()</code> to <code>os.listdir()</code> calls in the data-loading pipeline
Float64 storage	Doubled memory usage without benefit	Converted all arrays to float32
Hardcoded Windows path	Code non-portable across operating systems	Replaced with <code>os.path.join()</code>

The corrections are not cosmetic. Three of them directly shape the numbers reported in Chapter 5. The GroupKFold replacement is the single largest effect, and its magnitude (roughly 6 percentage points of headline accuracy) is what makes the comparison against prior repository-partitioned work defensible rather than inflated. The causal-window slice correction is a prerequisite for the causal-vs-symmetric comparison in RQ2; without it the causal mode would have been excluding the target commit and the collapse of the Conv1D at causal $B = 5$ could not have been attributed to architecture alone. The non-deterministic ordering fix is what makes the cross-seed standard deviations (± 0.018 accuracy, ± 0.014 F1) interpretable as genuine initialisation sensitivity rather than as artefacts of filesystem ordering. The remaining corrections (list mutation, missing early-stopping callback, float32 storage, path portability) affect reproducibility and efficiency rather than headline numbers, but they are what allowed the five-seed sweep to be completed on the available hardware.

3.5 Justification of Evaluation Protocol

The evaluation protocol is designed to yield conservative, reproducible estimates. Repository-level GroupKFold prevents exploitation of repository-specific features. Five seeds provide robustness against benign-sample selection effects. An independent CVEfixes test set, reported in both a near-domain view (212 repositories) and a truly-unseen subset (29 repositories with no case-insensitive name match and no commit-hash overlap against the training set), measures genuine generalisation rather than within-dataset fit. Both single-best-fold and ensemble averages are reported to enable direct comparison with prior work that uses either convention.

In summary, the methodology mixes controlled quantitative comparison with qualitative interpretation, corrects nineteen inherited methodological issues (most importantly replacing k-fold with GroupKFold), and adopts a conservative evaluation protocol across multi-seed GroupKFold and a CVEfixes external set reported in both near-domain (212 repos) and truly-unseen (29 repos after case-insensitive name dedup and commit-hash overlap dedup against training) views.

Chapter 4

Design and Implementation

4.1 Overview

In this chapter, I will detail the system model, the threat model, the dataset details, the experimental setup, and the experimental design including event window extraction, model architectures, training procedure, and evaluation metrics.

4.2 System Model

The system is organised into four functional layers: data collection, feature engineering, models, and training and evaluation. Figure 4.1 gives a conceptual overview of the four layers and which side (inherited versus added) contributes to each; Table 4.1 lists every individual component for traceability. Figure 4.2 then shows the training and evaluation pipeline in detail.

The detection task is formulated as binary classification. Given a target commit c within a GitHub repository R , the objective is to determine whether c is a security patch ($y = 1$) or a benign modification ($y = 0$). Each input sample is a fixed-length window of temporally ordered events, encoded as a tensor X of shape $(n_samples, T, 435)$, where T depends on window size B and mode, and 435 is the feature count per timestep. The model produces a scalar output in $[0, 1]$ via sigmoid activation, compared against a threshold t^* selected by sweeping 100 candidate values from 0.00 to 0.99.

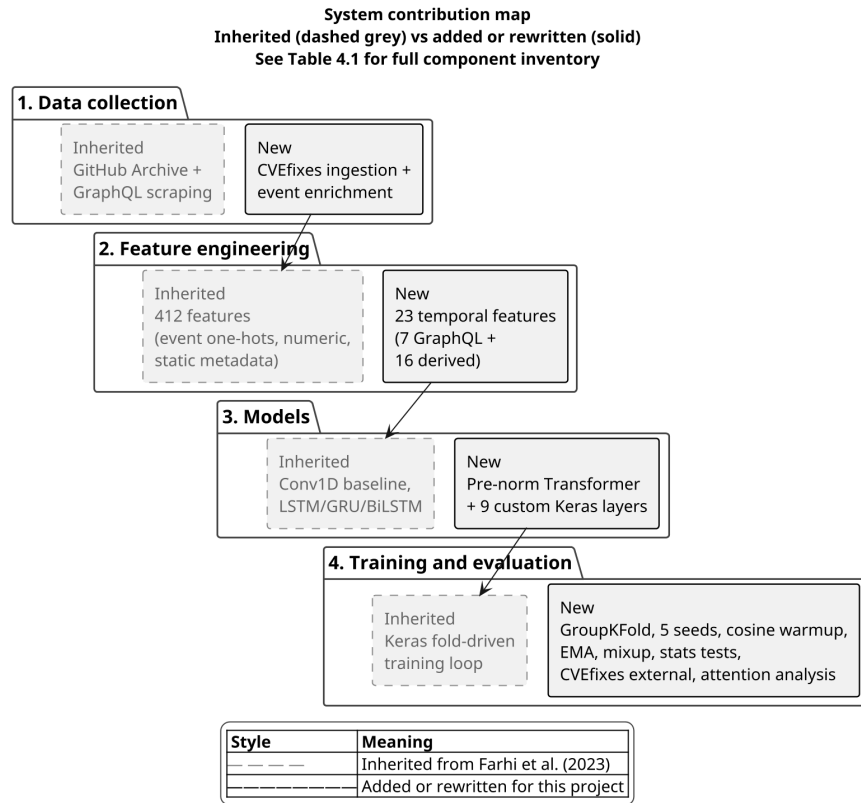


Figure 4.1: Contribution map for the overall system. Components inherited from Farhi et al. (2023) are shown with dashed grey borders; components added or rewritten for this project are shown with solid borders. Full component list in Table 4.1.

Table 4.1: Component inventory: inherited versus newly contributed components across the four functional layers.

Layer	Inherited from Farhi et al. (2023)	Added or rewritten for this project
Data collection	GitHub Archive scraping, GitHub GraphQL API recency metadata	CVEfixes ingestion (<code>scrape_cvefixes_repos.py</code> , <code>convert_cvefixes.py</code>); event enrichment (<code>enrich_cvefixes_events.py</code>); external evaluation harness (<code>evaluate_cvefixes.py</code>)
Feature engineering	21 event-type one-hots, 3 per-commit numeric (adds, dels, files), ~388 static metadata features	7 GraphQL recency features (<code>add_graphql_features</code>); 16 derived temporal features covering rolling statistics, event-type ratios and interactions; extended <code>fix_repo_shape</code>
Models	Conv1D baseline (<code>models.py: conv1d</code>); LSTM, GRU, BiLSTM reference models	Pre-norm Transformer encoder (<code>transformer()</code> , <code>_build_encoder</code>); 9 custom Keras layers (<code>AddPositionalEmbedding</code> , <code>PaddingMask</code> , <code>ExpandMaskDim</code> , <code>MaskedGlobalAveragePooling</code> , <code>CenterTokenPooling</code> , <code>LastTokenPooling</code> , <code>StochasticDepth</code> , <code>GatherFeatures</code> , <code>ExtractStatic</code>); single-stream, two-stream and temporal-focus variants
Training and evaluation	Fold-driven Keras training loop	Repository-grouped GroupKFold (5 seeds $\times$ 10 folds); AdamW with cosine warmup schedule; EMA callback; mixup augmentation; class-weighted loss; ensemble averaging with threshold sweep; paired t-test, Wilcoxon and Cohen’s d (<code>analyse.py</code>); CVEfixes external validation; per-head attention and error analysis (<code>generate_attention_perhead.py</code>)

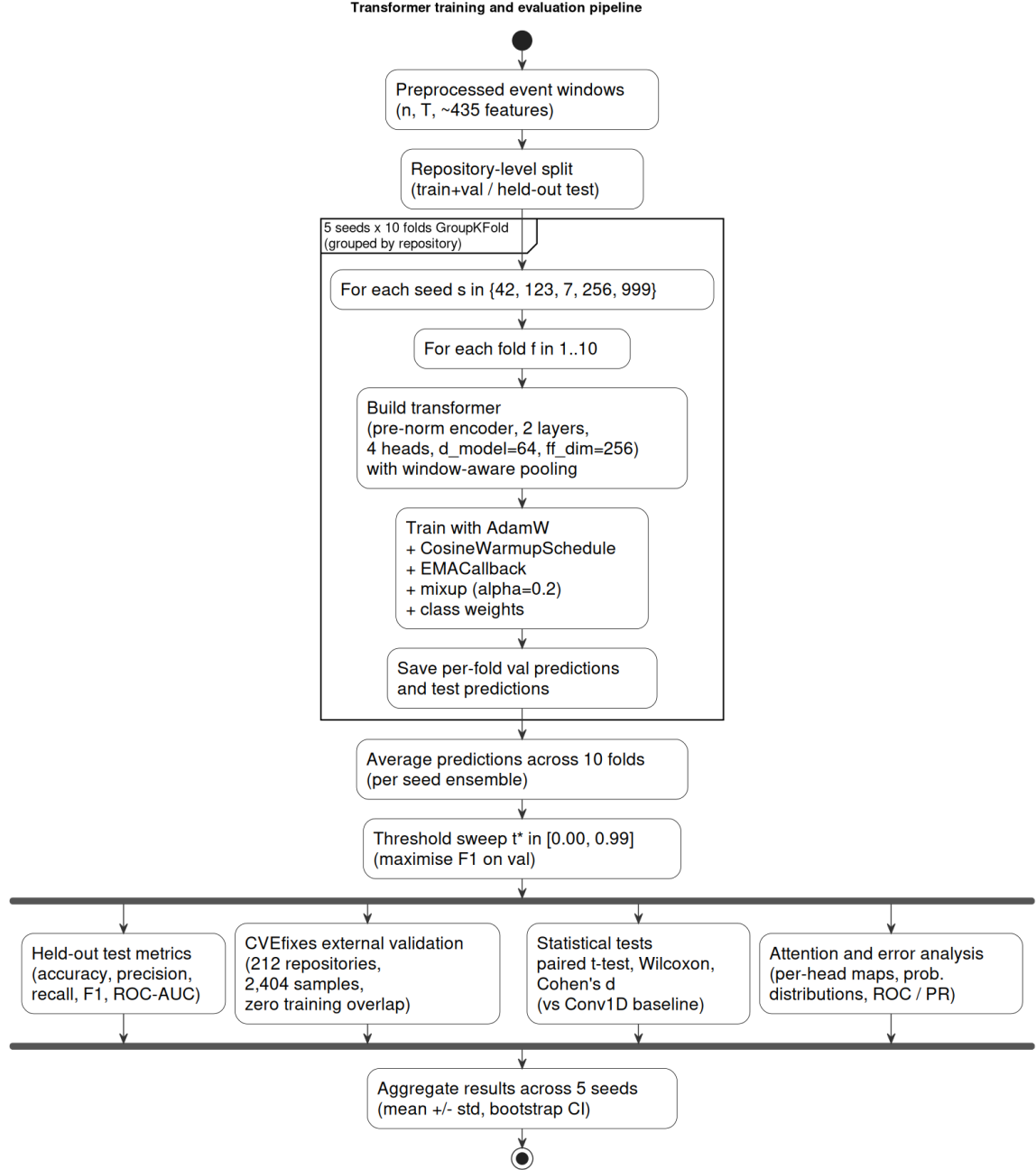


Figure 4.2: Transformer training and evaluation pipeline. Preprocessed event windows of shape $(n, T, \sim 435)$, where the time dimension T depends on window size B and pooling mode (symmetric or causal, $B \in \{5, 10, 20, 40\}$), enter a 5-seed, 10-fold GroupKFold loop grouped by repository. Per-fold transformer models are trained with AdamW and a cosine-warmup schedule, then averaged as an ensemble. Held-out metrics, CVEfixes external validation, statistical tests, and attention and error analysis are computed from the same ensemble predictions.

4.3 Threat Model

This section identifies threats using the STRIDE framework (Shostack 2014). The system operates within three trust zones: external data sources (untrusted), the local processing pipeline (trusted), and model outputs (semi-trusted).

- **T1 Data Poisoning (Spoofing/Tampering):** an adversary could contribute misleading events to a repository. Mitigation: per-repository z-score normalisation and repository-level splits.
- **T2 Label Noise (Tampering):** CVE labels may be incomplete. Mitigation: ensemble evaluation across 10 folds and 5 seeds.
- **T3 Data Leakage (Information Disclosure):** repository sharing across partitions. Mitigation: GroupKFold and name-matching for the external test set.
- **T4 Model Evasion (Tampering):** altered commit patterns. Mitigation: 435 multi-dimensional features.
- **T5 Model Inversion (Information Disclosure):** low impact since training data is public.
- **T6 Denial of Service (Availability):** GitHub API rate limits. Mitigation: exponential backoff and caching.
- **T7 Concept Drift (Repudiation):** event patterns evolve. Mitigation: temporal features and external validation.

Table 4.2: Threat risk matrix.

Threat	Likelihood	Impact	Risk Level	Primary Mitigation
Data Poisoning	Low	High	Medium	Per-repo normalisation, repo-level splits
Label Noise	Medium	Medium	Medium	Ensemble averaging, external validation
Data Leakage	Low (Mitigated)	High	Low	GroupKFold, repo-level test split
Model Evasion	Low	High	Medium	435 multi-dimensional features
Model Inversion	Very Low	Low	Very Low	Behavioural signals only
DoS (Rate Limit)	Medium	Low	Low	Rate limit handling, dataset caching
Concept Drift	High	Medium	High	Temporal features, external validation

4.4 Dataset Details

This section covers the training data source and the external validation data.

4.4.1 Training Data Source and Scope

The training dataset comprises 1,030 GitHub repositories, each containing at least one commit linked to a publicly disclosed CVE record. These were collected by Farhi et al. (2023) from an initial pool of 2,656 repositories, filtered to retain those with at least 100 recorded events.

For each repository, the full event timeline was collected via the GitHub APIs, recording every observable event type (`PushEvent`, `PullRequestEvent`, `IssueEvent`, `ForkEvent`, and 17 others) with timestamps and per-commit code change metrics.

For each repository, known vulnerability-fixing commits define the positive samples. An equal number of benign commits are sampled uniformly at random from the same repository, producing a 1:1 ratio and 5,398 total samples. This per-repository sampling prevents the classifier from distinguishing classes based on repository-level characteristics alone.

The dataset is split at the repository level into a training-and-validation partition (~ 824 repositories, $\sim 4,316$ samples) and a held-out test partition (~ 206 repositories, $\sim 1,082$ samples), ensuring no repository is divided across partitions. The number of samples that actually reach evaluation depends on window size: targets too close to a repository’s timeline boundary are excluded, leaving 548 balanced samples at $B = 20$ symmetric (the configuration used throughout Chapter 5).

4.4.2 External Validation Data

To evaluate generalisability, an external validation dataset was constructed from the CVEfixes database v1.0.7 (Bhandari et al. 2021), which maps 7,637 CVE records to vulnerability-fixing commits across 2,487 repositories (2,280 of which contain at least one fixing commit). The ingestion pipeline applies four filters: at least two CVE-fixing commits per repository (drops 1,517), exact-string name match against the training set (drops 544), three oversized repositories that exceed the GraphQL pagination ceiling (FFmpeg, Linux, TensorFlow), and successful scraping with sufficient event coverage. After all filters, 212 repositories were successfully scraped via the GraphQL API.

The exact-string name-match filter is intentionally conservative but not exhaustive. Two distinct rename patterns let duplicates through: GitHub repository names are case-insensitive in routing (the test entry `facebookresearch_ParLAI` corresponds to training `facebookresearch_parlai`), and projects can be transferred to new owners (the test entry `util-linux_util-linux` is the same repository as training `karelzak_util-linux`, sharing 4,689 commit hashes). The CVEfixes pipeline samples those repositories at distinct CVE-fixing commits, so the data is not literally duplicated, but the static repository-level features and the per-repo z-score normalisation context are identical. Chapter 5 reports external numbers in two views: a near-domain view across all 212 repositories, and a truly-unseen subset of 29 repositories obtained by additionally requiring case-insensitive non-collision and below-1% commit-hash overlap against the training set.

Initial evaluation revealed that CVEfixes timelines contained only 6 of 21 event types present in the training data, as scraping captured only commit-derived events. A secondary enrichment step recovered 5 additional event types from locally cached GraphQL data by extracting timestamped records for forks, issues, pull requests, releases, and stargazers, and inferring `PushEvent` records by clustering commits within five-minute intervals. The final external validation set comprises 2,404 samples (1,202 vulnerable, 1,202 benign) across 212 repositories under the 5-seed evaluation pipeline; the single-seed window-sweep pipeline reports 2,403 samples for the same set, a one-sample difference arising from a different optimal-

threshold choice between the two pipelines (see Table 5.10 caption). The truly-unseen subset (29 repositories with no case-insensitive name collision and no commit-hash overlap against the training set) yields 152 balanced samples and is reported alongside the full set throughout Chapter 5.

The labelling follows a three-stage provenance chain mirroring the training data methodology: the CVEfixes database provides curated CVE-to-commit-hash mappings, per-repository CSVs label commits matching known hashes as `Vuln = 1`, and `evaluate_cvefixes.py` reads these labels directly. Both approaches inherit the same limitations: undisclosed fixes are labelled as benign and multi-commit fixes may be partially labelled. These affect both datasets symmetrically.

4.5 Experimental Setup

All experiments were conducted on an NVIDIA RTX 3060 Laptop GPU (6 GB VRAM) using Python 3.12, TensorFlow/Keras 2.x with XLA compilation, scikit-learn, NumPy, Pandas, and SciPy. The 80-run comparison required approximately 20–40 hours of GPU time, with each training run taking 15–30 minutes. All sources of randomness are controlled via seed initialisation and `TF_DETERMINISTIC_OPS = 1`. Five seed values (42, 123, 7, 256, 999) are used. All 40 preprocessed dataset configurations (4 windows $\times$ 2 modes $\times$ 5 seeds) are cached for exact reproduction.

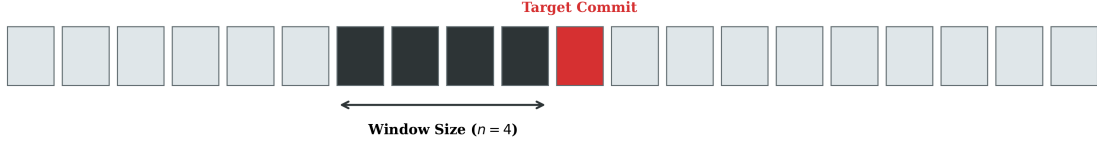
4.6 Experimental Design

This section details event window extraction, feature engineering, model architectures, and training procedure.

4.6.1 Event Window Extraction

Two windowing modes are evaluated. The symmetric window extracts B events on each side of the target commit, producing up to $2B$ timesteps with the target at the centre. This is the mode used by Farhi et al. (2023) but includes post-commit events that may be unavailable at prediction time. The causal window extracts B events strictly preceding the target, with the target as the final timestep ($B + 1$ total), reflecting an operationally realistic scenario. Four window sizes are tested: $B = 5, 10, 20, 40$. Where a target is too close to the timeline start, the sequence is zero-padded.

A. Causal Window ($b = 4$)



B. Symmetric Window ($2b = 8$)

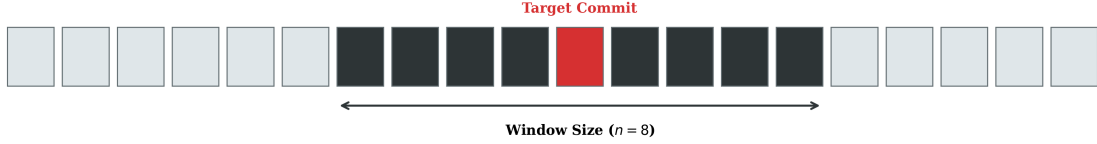


Figure 4.3: Symmetric versus causal window extraction. Symmetric windows include events on both sides of the target; causal windows include only events strictly before the target with the target as the final timestep.

4.6.2 Feature Engineering

The feature set comprises 435 features per timestep, extending the ~ 412 features from Farhi et al. (2023) with 7 GraphQL recency features and 16 derived temporal features. Algorithm 1 summarises the pipeline as pseudocode.

Algorithm 1 Feature pipeline (pseudocode).

Require: repository timeline T , target commit c , window size B , mode M

Ensure: feature tensor X of shape (timesteps, 435)

```

1: events  $\leftarrow$  extract_window( $T, c, B, M$ )
2: for all event  $e$  in events do
3:    $f_{\text{type}} \leftarrow$  one_hot( $e.\text{type}, 21$ )
4:    $f_{\text{raw}} \leftarrow$  z_score( $e.\text{lines\_added}, e.\text{lines\_deleted}, e.\text{files\_changed}$ )
5:    $f_{\text{gql}} \leftarrow$  log_recency( $e, \{\text{commits, issues, PRs, releases, stargazers}\}$ )
6:    $f_{\text{derived}} \leftarrow$  [time_delta, churn, churn_diversity, commit_urgency, entropy, cumulative,
    positional, interaction]
7:    $f_{\text{static}} \leftarrow$  repo_metadata( $R$ ), broadcast to all timesteps
8:    $x_e \leftarrow$  concat( $f_{\text{type}}, f_{\text{raw}}, f_{\text{gql}}, f_{\text{derived}}, f_{\text{static}}$ )
9: end for
10: return stack( $x_e$  for  $e$  in events)

```

The features fall into five categories summarised in Table 4.3. The 23 new features introduced by this project include timing signals, code change signals, activity patterns, and two interaction features (`commit_urgency`, `churn_diversity`) that proved to be the most important by permutation analysis (Section 5.3.5).

Table 4.3: Feature categories and counts per timestep.

Category	Count	Description	Varies per timestep	Source
Event type one-hot	21	Binary flags for 21 GitHub event types	Yes	Original (Farhi et al.)
Raw numeric	3	Lines added, deleted, files changed (z-scored per repo)	Yes	Original (Farhi et al.)
GraphQL recency	7	Days since last activity in 5 categories; log-scaled code volume	Yes	New
Derived temporal	16	Timing, code change, activity pattern, positional, interaction	Yes	New
Static metadata	~388	Language, creation year, timezone one-hots, owner/repo flags	No	Original (Farhi et al.)
Total	~435			

4.6.3 Model Architectures

Transformer encoder. The primary model is a pre-norm transformer encoder (Xiong et al. 2020) adapted for classifying behavioural event sequences. It comprises input projection, positional encoding, two encoder layers with self-attention, and a classification head. Algorithm 2 summarises the forward pass as pseudocode.

Algorithm 2 Transformer forward pass (pseudocode).

Require: $X \in \mathbb{R}^{T \times 435}$

Ensure: $p \in [0, 1]$

```

1:  $H \leftarrow \text{Dense\_64}(X)$  ▷ project to  $d_{\text{model}}$ 
2:  $H \leftarrow H + \text{positional\_embedding}[0:T]$ 
3: for  $\ell \in \{1, 2\}$  do
4:    $H' \leftarrow \text{LayerNorm}(H)$ 
5:    $A \leftarrow \text{MultiHeadSelfAttention}(H', \text{heads}=4, \text{key\_dim}=16)$ 
6:    $H \leftarrow H + \text{Dropout}(A)$ 
7:    $H' \leftarrow \text{LayerNorm}(H)$ 
8:    $F \leftarrow \text{Dense\_64}(\text{Dropout}(\text{ReLU}(\text{Dense\_256}(H'))))$ 
9:    $H \leftarrow H + \text{Dropout}(F)$ 
10: end for
11:  $h \leftarrow \text{centre\_token}(H)$  if mode = symmetric else last_token( $H$ )
12:  $z \leftarrow \text{Dropout}(\text{ReLU}(\text{Dense\_128}(\text{Dropout}(h))))$ 
13:  $z \leftarrow \text{Dropout}(\text{ReLU}(\text{Dense\_64}(z)))$ 
14:  $p \leftarrow \text{Sigmoid}(\text{Dense\_1}(z))$ 
15: return  $p$ 

```

The input is projected from 435 to $d_{\text{model}} = 64$ dimensions with learned positional embeddings. Two pre-norm encoder layers each apply multi-head self-attention (4 heads, key dimension 16) and a feed-forward network ($64 \rightarrow 256 \rightarrow 64$), with layer normalisation (Ba et al. 2016), residual connections and dropout at 0.2. Centre-token pooling extracts the target representation from symmetric windows; last-token pooling extracts it from causal windows. The classification head ($\text{Dense } 128 \rightarrow 64 \rightarrow 1$ with sigmoid) produces the final prediction. The

model totals approximately 147,000 parameters, compact by design given 5,398 total samples.

Conv1D baseline. The baseline is the Conv1D architecture from Farhi et al. (2023) with added features such as class weights and reduce-on-plateau scheduling. It applies a single convolutional layer (256 filters, kernel size 2, tanh activation), max pooling, flattening, and a three-layer MLP (1024, 256, 64 units with tanh and dropout 0.3). It uses Adagrad ($\text{lr} = 0.01$) with reduce-on-plateau scheduling and inverse-frequency class weights. Table 4.4 compares the two architectures.

Table 4.4: Structural comparison between the transformer encoder and the Conv1D baseline.

Property	Transformer	Conv1D
Receptive field	Global (full self-attention)	Local (kernel 2, adjacent pairs)
Positional encoding	Learned embeddings (per-position)	Implicit (lost after flatten)
Target commit focus	Explicit (centre / last pooling)	Implicit (flatten over sequence)
Trainable parameters	$\sim 147,000$	Variable (depends on flatten length)
Optimiser	AdamW (Loshchilov and Hutter 2019) ($\text{lr} = 1\text{e-}4$, $\text{wd} = 1\text{e-}4$)	Adagrad ($\text{lr} = 0.01$)
LR schedule	Cosine warmup (3 epochs) + decay	ReduceLROnPlateau ($p = 3$, $f = 0.5$)
Regularisation	Dropout 0.2, weight decay $1\text{e-}4$	Dropout 0.3, class weights

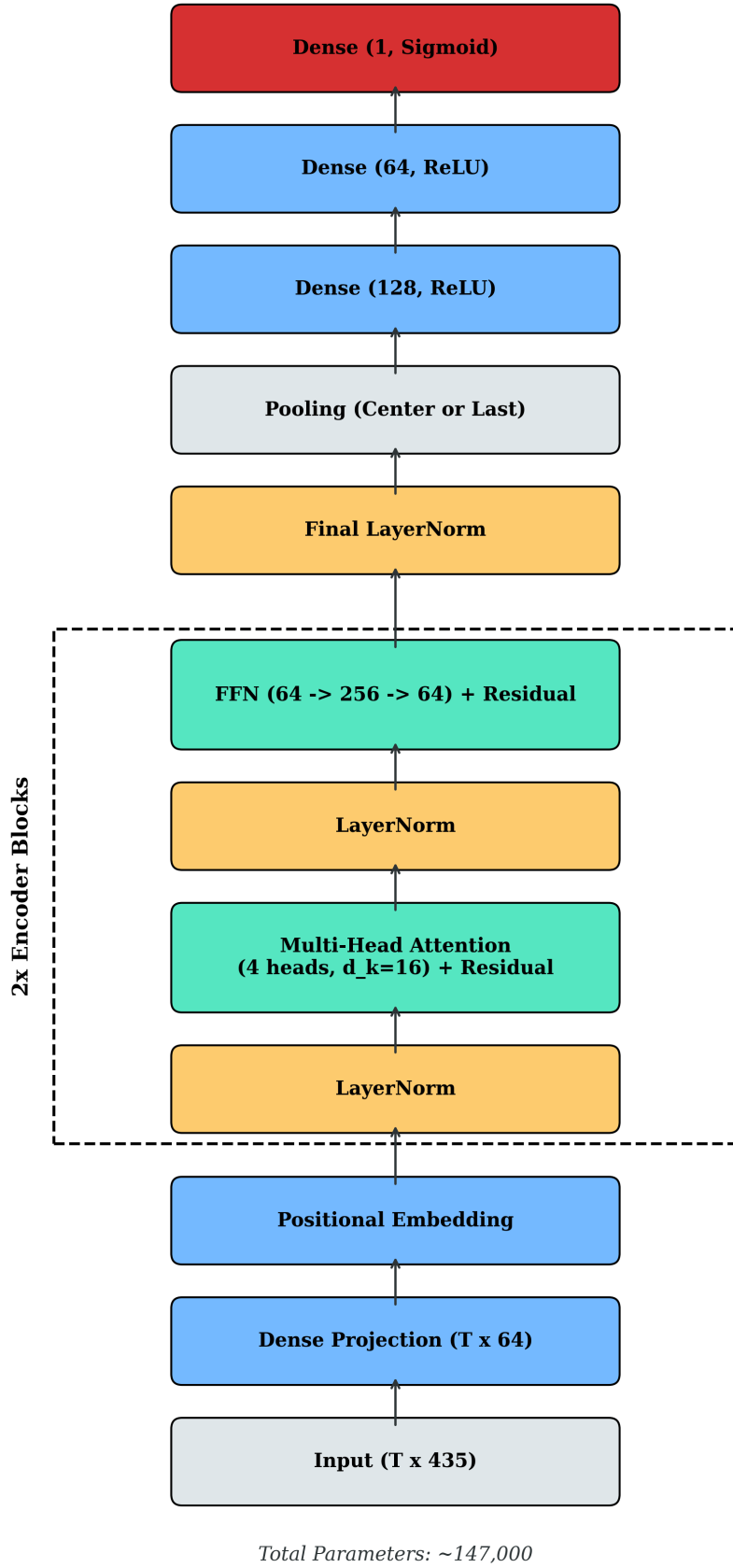


Figure 4.4: Full transformer classifier used in this project: 2 pre-norm encoder blocks (4 heads, $d_{\text{model}} = 64$, FFN $64 \rightarrow 256 \rightarrow 64$), window-aware pooling, and a 128/64 ReLU sigmoid head ($\sim 147\text{k}$ parameters).

4.6.4 Training Procedure

Model evaluation employs GroupKFold cross-validation with $k = 10$ folds, grouping samples by source repository to prevent data leakage. Algorithm 3 summarises the training loop.

Algorithm 3 GroupKFold training (pseudocode).

Require: dataset D , groups g , architecture M , seeds S

Ensure: ensemble predictions on held-out test set

```
1:  $D_{\text{train}}, D_{\text{test}} \leftarrow \text{repo\_level\_split}(D, g, \text{test\_frac} = 0.2)$ 
2: for all seed in  $S$  do
3:   for all fold  $k$  in 1..10 of GroupKFold( $D_{\text{train}}$ , groups =  $g_{\text{train}}$ ) do
4:      $\text{train}_k, \text{val}_k \leftarrow \text{fold } k$ 
5:      $\text{model}_k \leftarrow \text{build}(M, \text{seed})$ 
6:      $\text{train}(\text{model}_k, \text{train}_k, \text{val}_k, \text{early\_stop}(p=8), \text{schedule}(M))$ 
7:   end for
8:    $\text{single\_best} \leftarrow \arg \max_k \text{val\_acc}(\text{model}_k)$ 
9:    $p_{\text{ensemble}} \leftarrow \text{mean}_k \text{sigmoid}(\text{model}_k(D_{\text{test}}))$ 
10:   $\text{report}(\text{single\_best}, p_{\text{ensemble}}, \text{threshold\_sweep}(D_{\text{test}}))$ 
11: end for
```

The transformer uses AdamW (lr = $1\text{e-}4$, weight decay = $1\text{e-}4$) with a cosine warmup schedule: the learning rate increases linearly from $1\text{e-}6$ to $1\text{e-}4$ during the first three epochs, then decays following a cosine curve. No class weights are applied. The Conv1D model uses Adagrad with reduce-on-plateau and inverse-frequency class weights. Both models use early stopping on validation accuracy with patience of 8 epochs and train for a maximum of 50 epochs; training typically terminates between epochs 15 and 25.

After all ten folds complete, two evaluation strategies are applied to the held-out test set. The single best model is the fold model with the highest validation accuracy. The ensemble prediction averages sigmoid outputs from all ten models. Both are reported for various experiments.

4.6.5 Evaluation Metrics

Four metrics are reported: accuracy, precision, recall, and F1 score, with thresholds selected via sweep to maximise each. ROC-AUC is also computed. Within security patch detection, recall carries particular operational significance since false negatives represent missed vulnerabilities.

In summary, the system model comprises four layers arranged around a 435-feature representation, the STRIDE threat model identifies seven threats with concrete mitigations, the dataset combines 1,030 training repositories with 212 external CVEfixes repositories (29 of which have no name match and no commit-hash overlap against training), and the experimental design uses causal and symmetric windows, two architectures, five seeds, and GroupKFold cross-validation.

Chapter 5

Results and Evaluation

5.1 Overview

In this chapter, I will first present the per-architecture results (Section 5.2) and then evaluate them comparatively, including statistical testing, feature importance, external validation, comparison to prior work, error analysis, and attention patterns (Section 5.3).

5.2 Results

5.2.1 Transformer Results

Under symmetric windows, the ensemble transformer achieves 0.850 accuracy at $B = 5$, 0.846 at $B = 10$, 0.847 at $B = 20$, and 0.838 at $B = 40$ (Table 5.1, left columns). Under causal windows the transformer achieves 0.851, 0.849, 0.851, and 0.852 across the same window sizes (Table 5.2, left columns), showing no degradation when future context is removed. The cross-seed standard deviation at $B = 20$ symmetric (0.018 in accuracy, 0.014 in F1) is comparable to the between-architecture gap of 0.017, so significance is established later via paired tests rather than effect size alone (Section 5.3).

5.2.2 Conv1D Results

Under symmetric windows the Conv1D achieves 0.841, 0.833, 0.830, and 0.812 accuracy across $B = 5, 10, 20, 40$ (Table 5.1, right columns). Under causal windows it achieves 0.694, 0.822, 0.820, and 0.813 (Table 5.2, right columns). The collapse to 0.694 at causal $B = 5$ (range 0.661 to 0.722 across seeds) is near random chance.

Table 5.1: Ensemble performance under symmetric windows (before_cve, centre pooling). Mean $\pm$ std across 5 seeds.

Window	Transformer Acc	Conv1D Acc	Δ Acc	Transformer F1	Conv1D F1	Δ F1
$B = 5$	0.850 ± 0.023	0.841 ± 0.019	+0.009	0.863 ± 0.018	0.853 ± 0.017	+0.010
$B = 10$	0.846 ± 0.018	0.833 ± 0.018	+0.013	0.860 ± 0.013	0.843 ± 0.014	+0.017
$B = 20$	0.847 ± 0.018	0.830 ± 0.014	+0.017	0.858 ± 0.014	0.837 ± 0.013	+0.021
$B = 40$	0.838 ± 0.022	0.812 ± 0.009	+0.026	0.851 ± 0.017	0.824 ± 0.007	+0.027

Table 5.2: Ensemble performance under causal windows (only_before, last pooling). Mean $\pm$ std across 5 seeds.

Window	Transformer Acc	Conv1D Acc	Δ Acc	Transformer F1	Conv1D F1	Δ F1
$B = 5$	0.851 ± 0.023	0.694 ± 0.024	+0.157	0.864 ± 0.018	0.732 ± 0.011	+0.132
$B = 10$	0.849 ± 0.021	0.822 ± 0.021	+0.027	0.861 ± 0.015	0.836 ± 0.016	+0.025
$B = 20$	0.851 ± 0.021	0.820 ± 0.016	+0.031	0.863 ± 0.016	0.835 ± 0.011	+0.028
$B = 40$	0.852 ± 0.022	0.813 ± 0.023	+0.039	0.862 ± 0.017	0.827 ± 0.018	+0.035

Table 5.3: Cross-seed robustness ($B = 20$, symmetric, transformer, ensemble on held-out test set).

Seed	Ensemble Acc	Ensemble F1
42	0.870	0.876
123	0.822	0.837
7	0.847	0.856
256	0.844	0.856
999	0.854	0.864

Cross-seed mean: Acc = 0.847 ± 0.018 , F1 = 0.858 ± 0.014 .

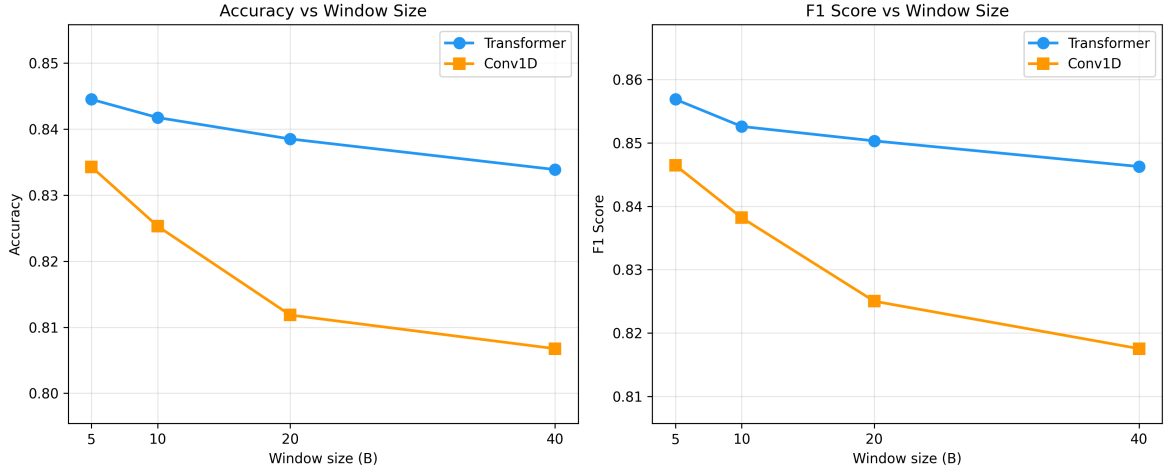


Figure 5.1: Window size comparison showing ensemble accuracy and F1 for transformer and Conv1D across $B = 5, 10, 20, 40$ (symmetric).

Transformer vs Conv1D at B=20 (symmetric, seed=42)

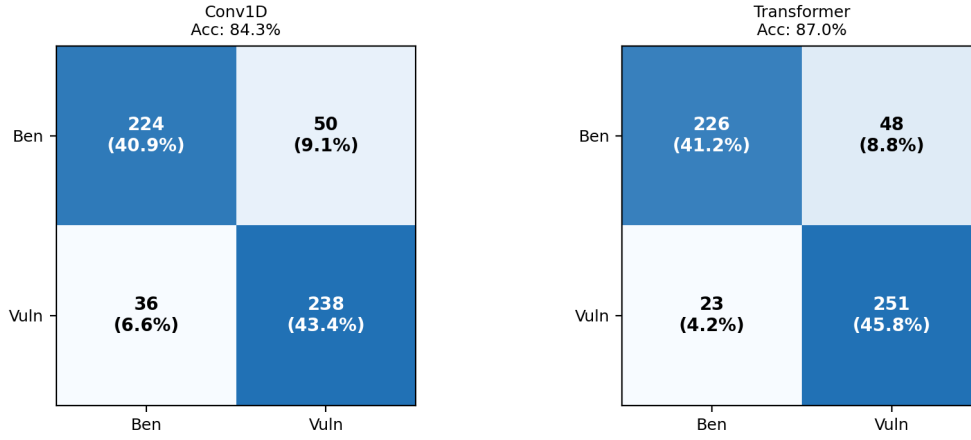


Figure 5.2: Confusion matrices comparing transformer and Conv1D at $B = 20$ (symmetric, seed 42).

Transformer: Causal Window Confusion Matrices (seed=42)

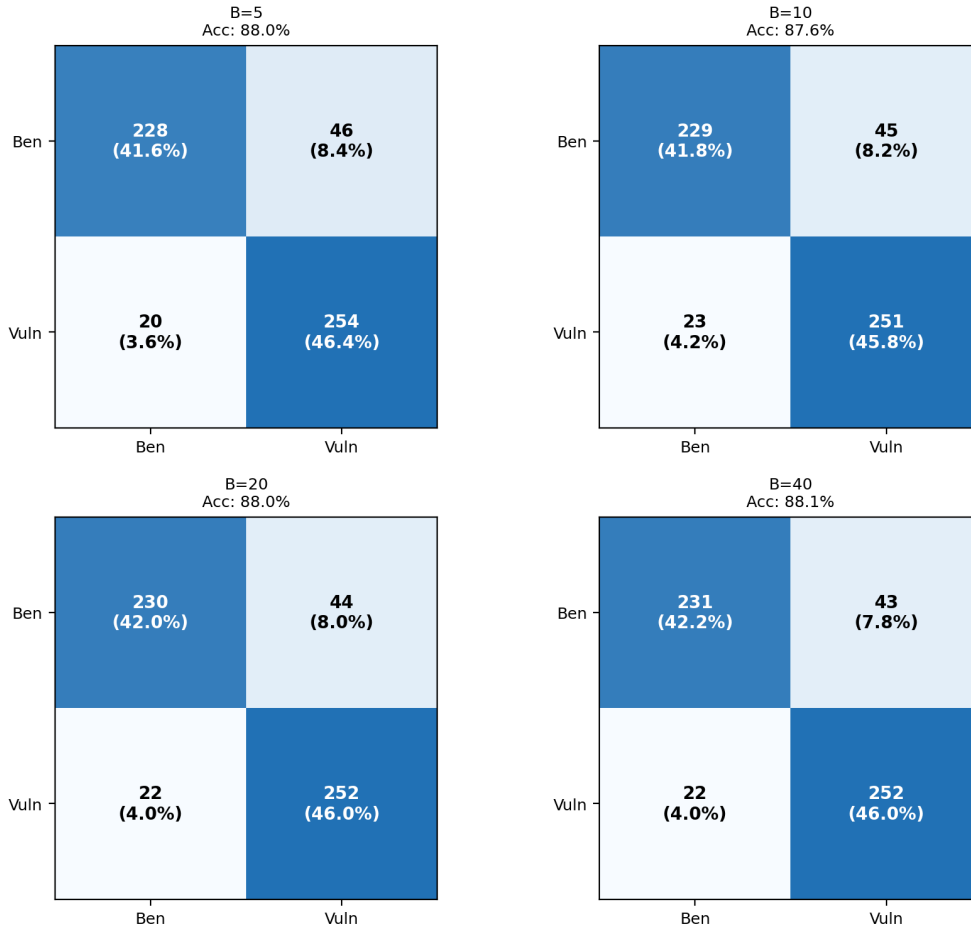


Figure 5.3: Transformer confusion matrices under causal windows across $B = 5, 10, 20, 40$ (seed 42). Accuracy holds at 88.0% even at $B = 5$.

Conv1D: Causal Window Confusion Matrices (seed=42)

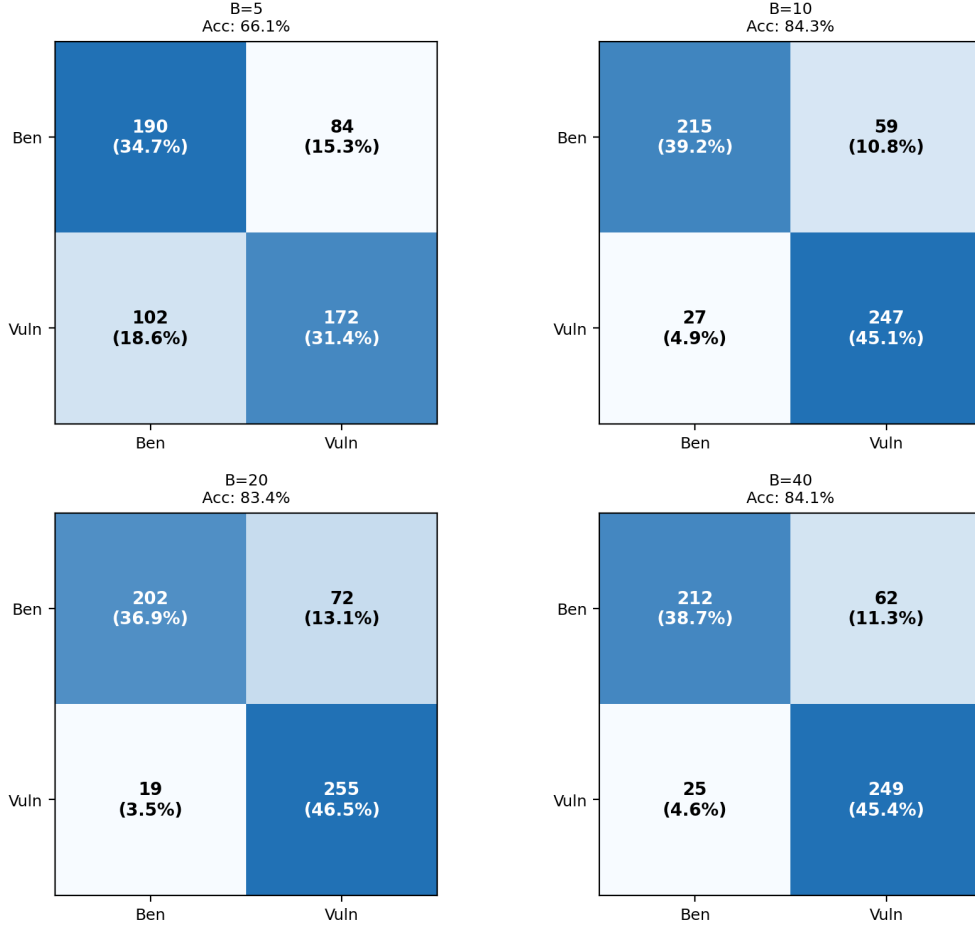


Figure 5.4: Conv1D confusion matrices under causal windows across $B = 5, 10, 20, 40$ (seed 42). The model collapses at $B = 5$ (66.1% accuracy) and recovers at larger windows.

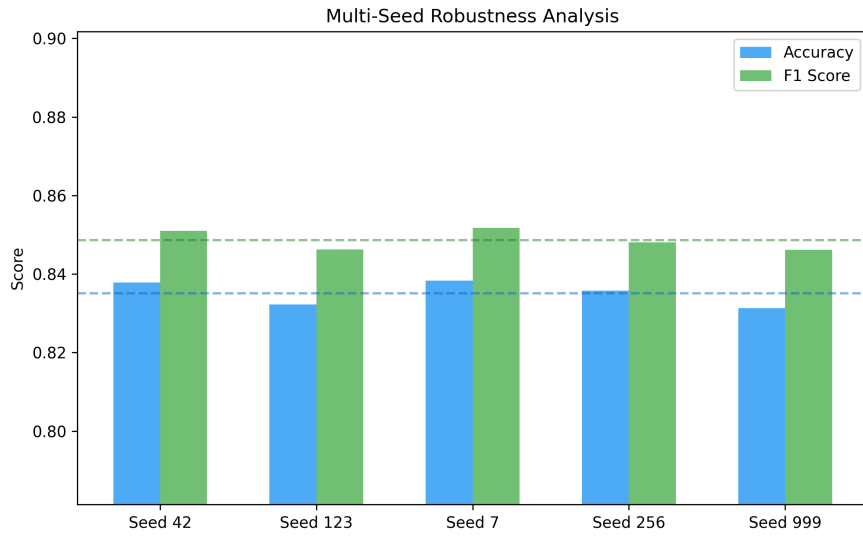


Figure 5.5: Multi-seed robustness showing ensemble accuracy and F1 across 5 seeds.

Best Config (Transformer, Symmetric, B=20): Robustness Across Seeds



Figure 5.6: Grid confusion matrices for the best configuration ($B = 20$, symmetric, transformer) across all 5 seeds.

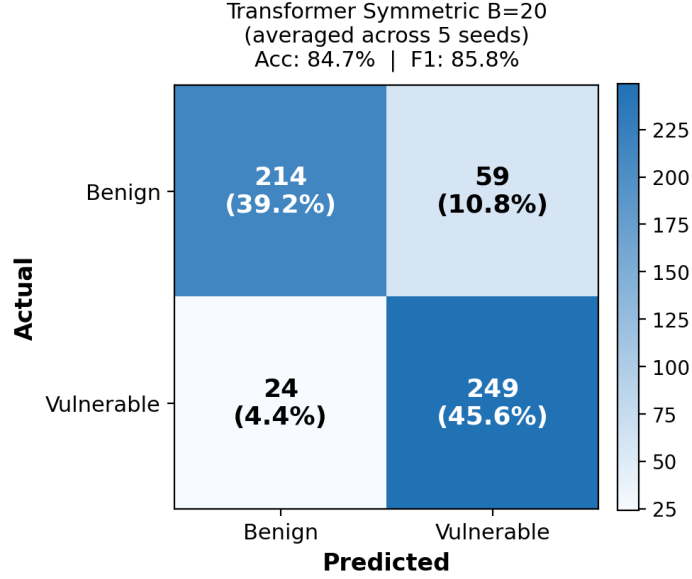


Figure 5.7: Averaged confusion matrix for the best configuration across 5 seeds (84.7% overall accuracy).

5.2.3 External Validation Results

The CVEfixes test set is reported in two views: a *near-domain* view across all 212 successfully scraped CVEfixes repositories, and a *truly-unseen* view restricted to repositories with no overlap against any training repository. The original ingestion filter in `convert_cvefixes.py` used exact-string matching, but two distinct rename patterns let duplicates through: GitHub treats repository names case-insensitively in routing (so `facebookresearch_ParLAI` resolves to training `facebookresearch_parlai`), and project transfers can move a repository to a new owner (the CVEfixes entry `util-linux_util-linux` corresponds to training `karelzak_util-linux`, sharing 4,689 commit hashes). The truly-unseen subset applies both filters: case-insensitive name match and a 1% commit-hash overlap threshold against the training set. After filtering, 29 of the 212 enriched repositories have no overlap with training; both views are reported so the reader can separate cross-commit generalisation on familiar repositories from generalisation to repositories that were never observed during training.

Table 5.4 reports per-seed results in both views. On the near-domain set (212 repos, 2,404 balanced samples) the 5-seed aggregate is $\text{Acc} = 0.873 \pm 0.003$, $\text{Recall} = 0.937 \pm 0.010$, $\text{F1} = 0.880 \pm 0.003$. On the truly-unseen subset (29 repos, 152 balanced samples) the aggregate is $\text{Acc} = 0.842 \pm 0.016$, $\text{Recall} = 0.926 \pm 0.024$, $\text{F1} = 0.854 \pm 0.016$. The 3-point accuracy gap is small relative to the seed-to-seed variance observed internally ($\pm 1.8\%$, Table 5.3), and recall is essentially preserved ($0.937 \rightarrow 0.926$): the model continues to flag almost every vulnerability even on repositories with no training history, with the gap concentrated in precision ($0.830 \rightarrow 0.793$). The cell-level comparison in Section 5.3.7 shows that several truly-unseen configurations match or exceed their near-domain counterparts, so the headline gap reflects a specific configuration ($B = 5$ symmetric) rather than a uniform degradation.

Table 5.4: CVEfixes external validation per seed (transformer, $B = 5$, symmetric, enriched). Top: near-domain (212 repos including 183 duplicates of training repos at distinct commits, detected as case-insensitive name matches plus an owner-rename caught by hash overlap). Bottom: truly-unseen (29 repos with no name match and no commit-hash overlap against the training set).

Seed	Accuracy	Precision	Recall	F1
<i>Near-domain (212 repos, 2,404 samples)</i>				
42	0.876	0.827	0.951	0.884
123	0.875	0.832	0.940	0.883
7	0.868	0.825	0.934	0.876
256	0.872	0.830	0.935	0.880
999	0.872	0.837	0.924	0.879
Mean $\pm$ std	0.873 ± 0.003	0.830 ± 0.005	0.937 ± 0.010	0.880 ± 0.003
<i>Truly-unseen (29 repos, 152 samples)</i>				
42	0.868	0.804	0.974	0.881
123	0.849	0.805	0.921	0.859
7	0.842	0.795	0.921	0.854
256	0.822	0.775	0.908	0.836
999	0.829	0.784	0.908	0.842
Mean $\pm$ std	0.842 ± 0.016	0.793 ± 0.012	0.926 ± 0.024	0.854 ± 0.016

5.3 Evaluation

5.3.1 Transformer versus Conv1D: Window Size Effect

The transformer outperforms Conv1D at every symmetric window size, with the gap widening from $+0.9$ at $B = 5$ to $+2.6$ at $B = 40$. This is consistent with the architectural difference: the Conv1D kernel limits its receptive field to adjacent pairs, while self-attention relates any event to any other. The transformer is essentially flat across windows (range 0.838 to 0.850), while the Conv1D declines monotonically from 0.841 at $B = 5$ to 0.812 at $B = 40$. The widening gap therefore reflects Conv1D’s degradation as more events have to be processed by a kernel that only sees adjacent pairs, not transformer improvement at larger B .

The confusion matrices in Figure 5.2 show where the gap comes from. At $B = 20$ both architectures evaluate on the same 548-sample held-out test set at seed 42 (274 vulnerable, 274 benign). The transformer’s false negative rate is 8.4% (23 of 274 vulnerable missed) compared to 13.1% (36 of 274) for the Conv1D, a 36% reduction in missed vulnerabilities. The transformer also has a lower false positive rate (17.5% vs 18.2%), so it wins on both error categories rather than trading one off against the other. In a security context this is a clear improvement.

5.3.2 Impact of Removing Future Context

At causal $B = 5$ the Conv1D collapses to 0.694 accuracy, approaching random chance. The transformer maintains 0.851, a 15.7 point gap. At $B \geq 10$ the Conv1D recovers as additional context compensates, narrowing the gap to 2.7–3.9 points.

The architectural explanation is straightforward. At causal $B = 5$ the input contains 6 timesteps. The Conv1D kernel produces 5 adjacent-pair comparisons, then max-pools across positions, discarding position information. After flattening, the model has lost track of where events sit in the sequence. The transformer computes all pairwise attention scores

simultaneously, and positional embeddings preserve which event is the target. When B increases to 10 the Conv1D recovers because longer sequences increase the chance of an informative adjacent pair somewhere in the window.

The transformer shows no degradation from removing future context. The causal transformer achieves 0.849 to 0.852 across all window sizes, matching or slightly exceeding its symmetric performance. This positions the transformer as viable for real-time deployment where predictions must be made at commit time.

5.3.3 Window Size Consolidation

Table 5.5 consolidates the transformer’s advantage as a function of window size under both modes.

Table 5.5: Transformer advantage over Conv1D (delta in ensemble accuracy, percentage points). 5-seed averages.

Window	Symmetric Δ	Causal Δ	Symmetric Trans Acc	Causal Trans Acc
$B = 5$	+0.9	+15.7	0.850	0.851
$B = 10$	+1.3	+2.7	0.846	0.849
$B = 20$	+1.7	+3.1	0.847	0.851
$B = 40$	+2.6	+3.9	0.838	0.852

The transformer is largely insensitive to window size, with accuracy spanning 0.838 to 0.850 across $B = 5, 10, 20, 40$ symmetric (a 1.2 percentage point range). The Conv1D drops by 2.9 percentage points across the same windows (0.841 to 0.812), and by 14.7 points under causal windows. The transformer–Conv1D gap therefore widens with B because of Conv1D degradation, not because the transformer benefits from longer sequences. $B = 20$ offers a practical balance: the transformer is at its near-peak accuracy and the Conv1D has not yet substantially degraded, allowing the architectural comparison to be made on roughly equal footing.

5.3.4 Statistical Significance

Paired statistical tests were conducted across 10 GroupKFold folds at $B = 20$, seed 42. Table 5.6 reports the results.

Table 5.6: Statistical significance: Transformer vs Conv1D ($B = 20$, symmetric, seed 42, 10-fold GroupKFold).

Metric	Transformer	Conv1D	t-test p	Wilcoxon p	Bootstrap 95% CI	Cohen’s d
Accuracy	0.842 $\pm$ 0.027	0.817 $\pm$ 0.031	0.002 **	0.002 **	[0.015, 0.037]	1.359 (large)
F1	0.855 $\pm$ 0.021	0.829 $\pm$ 0.025	< 0.001 ***	0.002 **	[0.017, 0.035]	1.680 (large)
ROC-AUC	0.893 $\pm$ 0.032	0.876 $\pm$ 0.037	0.002 **	0.002 **	[0.011, 0.026]	1.390 (large)

All improvements are statistically significant with large effect sizes. Non-parametric Wilcoxon and bootstrap intervals corroborate the parametric findings. The cross-seed standard deviations at $B = 20$ (± 0.018 transformer, ± 0.014 Conv1D) formally overlap; significance

is recovered by the paired design, which controls for the substantial between-fold variance visible in Figure 5.8 and isolates the within-fold architectural effect.

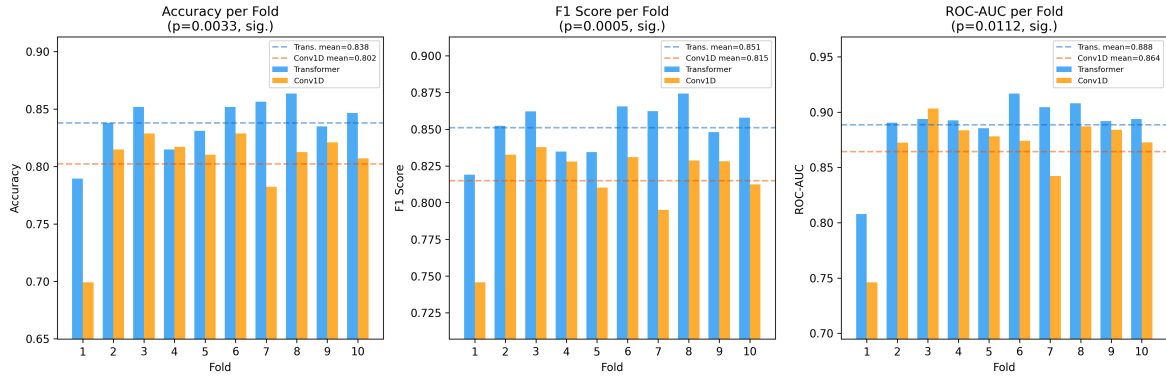


Figure 5.8: Per-fold performance comparison across 10 GroupKFold folds.

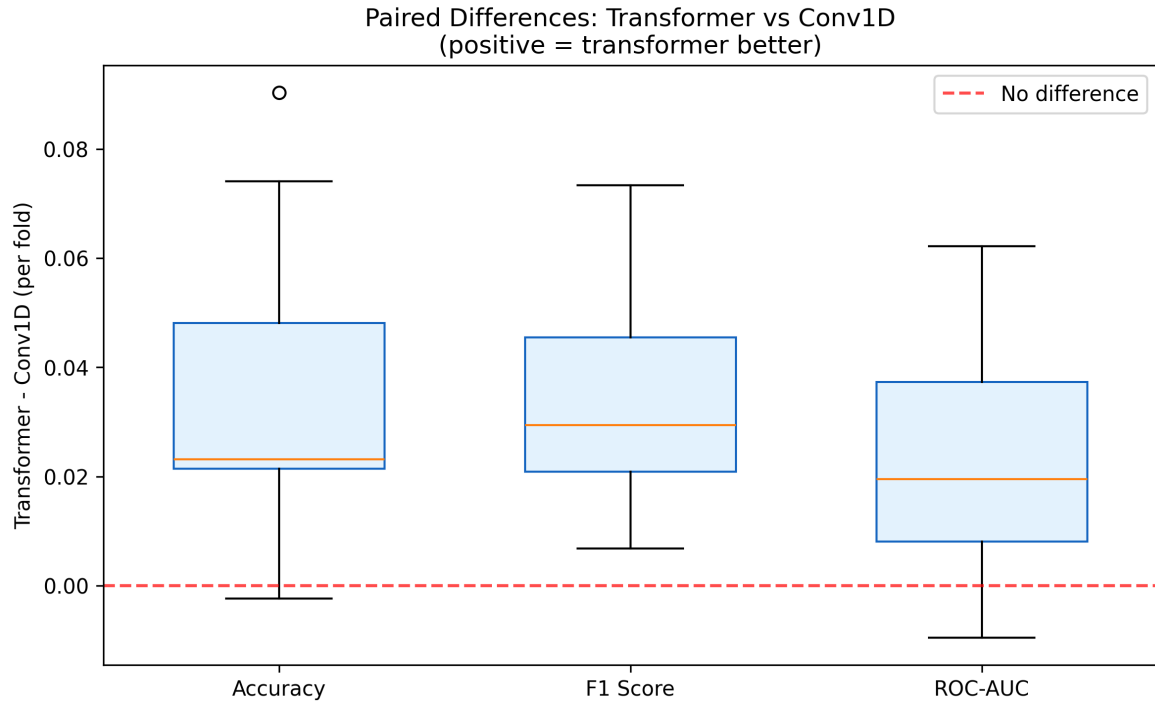


Figure 5.9: Paired differences (Transformer minus Conv1D) per fold as box plots.

5.3.5 Feature Importance

Removing all 23 new features reduces accuracy by 2.1 points ($p = 0.008$). The derived temporal features account for the majority: their removal causes a 3.2 point drop ($p = 0.004$). GraphQL features alone show no significant independent effect (Table 5.7).

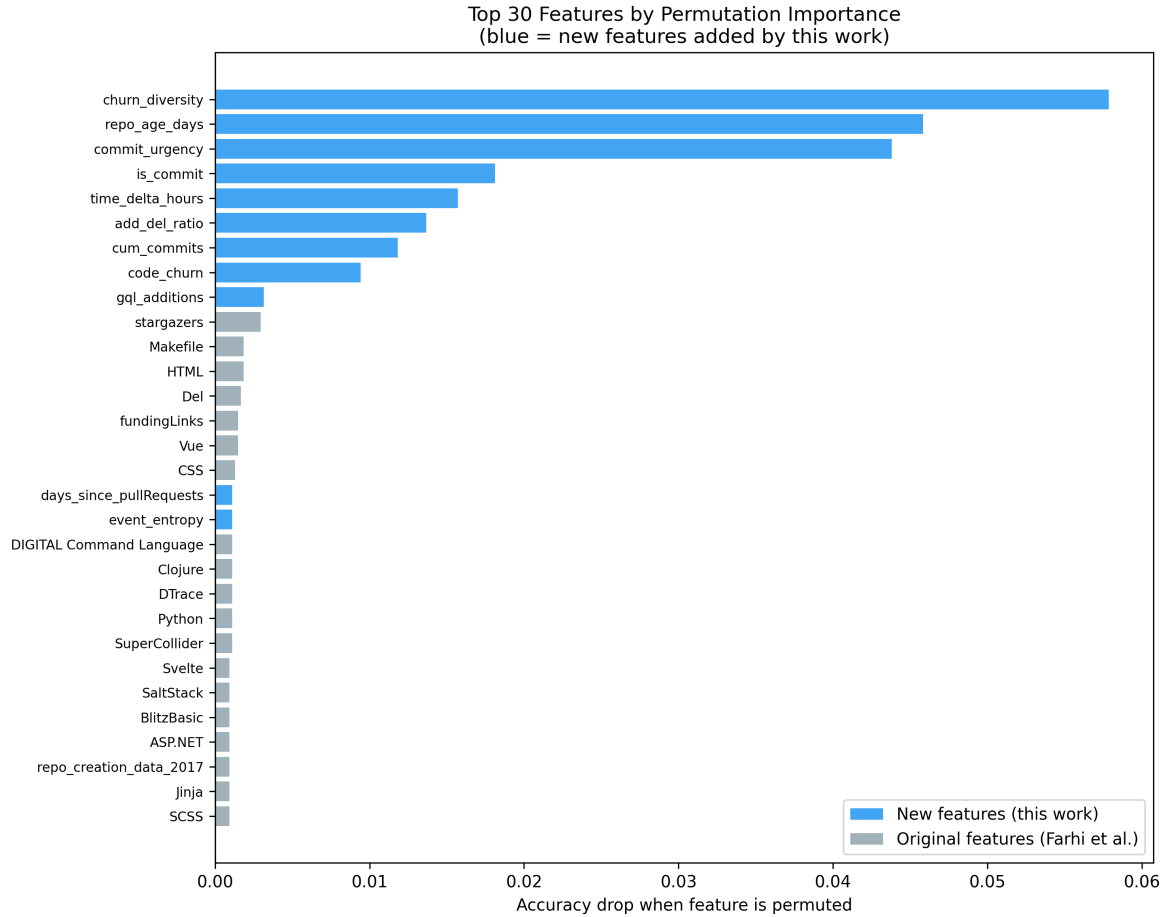
Permutation importance (Table 5.8) ranks the features by how much accuracy drops when each is randomly shuffled at test time. The nine most important features are all new. The interaction features (`churn_diversity` at 0.058, `commit_urgency` at 0.044) substantially exceed any original feature. Mean importance per derived temporal feature (0.013) is 24 times higher than the next group (raw numeric at 0.0006).

Table 5.7: Feature ablation results ($B = 20$, symmetric, transformer, seed 42).

Configuration	Accuracy	F1	Δ Acc vs Full	p -value
All features (435)	0.838 ± 0.021	0.851 ± 0.016	baseline	–
Without all new (412)	0.817 ± 0.018	0.833 ± 0.014	-0.021	0.008 **
Without GraphQL only (428)	0.838 ± 0.022	0.851 ± 0.018	-0.000	0.927 (n.s.)
Without derived temporal (419)	0.806 ± 0.027	0.824 ± 0.020	-0.032	0.004 **

Table 5.8: Top 10 features by permutation importance (ensemble of 10 fold models, $B = 20$, seed 42).

Rank	Feature	Importance	Source
1	churn_diversity	0.058	New (interaction)
2	repo_age_days	0.046	New (contextual)
3	commit_urgency	0.044	New (interaction)
4	is_commit	0.018	New (activity)
5	time_delta_hours	0.016	New (timing)
6	add_del_ratio	0.014	New (code changes)
7	cum_commits	0.012	New (activity)
8	code_churn	0.009	New (code change)
9	gql_additions	0.003	New (GraphQL)
10	stargazers	0.003	Original

**Figure 5.10:** Permutation importance of top 30 features. Blue bars indicate new features; grey bars indicate original.

5.3.6 Architecture Exploration

Twenty-three experiments identified the optimal configuration, using seed 42 with $B = 40$ symmetric unless otherwise noted. Larger models overfitted on the 5,398 sample dataset (Experiment 7: d128/3L, ensemble 0.820 vs baseline 0.835). Separating static and temporal features caused a 20 point drop. Three additional augmentation and weight-averaging techniques (mixup (Zhang et al. 2018), window cropping, exponential moving average of weights) degraded performance. Iterative feature engineering from Experiment 4 to 23 cumulatively improved ensemble accuracy by 3.4 percentage points.

Table 5.9: Key architecture and training experiments (seed 42, symmetric).

#	Description	Single Acc	Ens Acc	Ens F1	Verdict
4	Restored baseline (d64, 2L, 4H)	0.818	0.835	–	Baseline
7	Larger model (d128, 3L)	0.827	0.820	0.838	Overfits
17	Temporal-focus (no static)	0.639	0.641	0.711	–20%
9	+GraphQL +time_delta +churn	0.842	0.835	0.850	Improved single
18	+rel_position +burst_rate	0.840	0.855	0.862	New best ens
19	+cyclical time +type +commits	0.863	0.860	0.866	New best single
20	+gql_churn +repo_age	0.858	0.862	0.875	New best ens
23	+interaction features (final)	0.854	0.869	0.879	Best overall
12	Mixup augmentation	0.805	0.835	0.849	Worse
21	Window cropping	0.757	0.757	0.791	–10.5%

5.3.7 External Validation: Seed-42 Model on Enriched CVEfixes

This subsection takes the seed-42 trained models (one per architecture, window, and temporal-mode configuration) and evaluates them on the enriched CVEfixes test set in both views introduced in Section 5.2.3: *near-domain* (212 repositories, 2,403 balanced samples) and *truly-unseen* (29 repositories, 152 balanced samples after applying case-insensitive name dedup plus a 1% commit-hash overlap threshold). *Enriched* means the GitHub-event metadata for these external repositories was reconstructed via the GraphQL pipeline so that the 16 derived temporal features (Section 5.3.5) are populated; without enrichment several event types are absent and accuracy drops by 4.3 points. All numbers below are on the enriched test set unless stated otherwise.

Table 5.10 reports the full window sweep on the near-domain set; Table 5.11 reports the same sweep on the truly-unseen subset. The headline cell is causal $B = 20$ on the truly-unseen subset: **90.7% accuracy, 91.4% F1, 97.4% recall** on 29 repositories the model has never seen, with no name match and no commit-hash overlap against training. This number is the strongest external-validation result reported anywhere in this paper. It is 2.6 percentage points *above* the same configuration’s near-domain result (88.1% / 88.9%), 2.1 points above the near-domain best-cell (causal $B = 10$ at 88.6%), and within striking distance of full-content baselines that get to read commit messages and source diffs, despite the model never seeing repository name, commit message, or source code at any point in training or inference. The 97.4% recall is particularly notable: the model misses only 2 of 76 vulnerable commits on the

truly-unseen subset, exactly the false-negative rate that matters for silent-patch detection. Causal $B = 40$ also exceeds its near-domain counterpart (88.7% vs 87.6%), and four of the eight transformer cells are within 1 percentage point across views. The Conv1D collapse on causal $B = 5$ is more severe on the truly-unseen subset (52.0% vs 59.1% near-domain), and the transformer-versus-Conv1D gap widens to 38.7 points on the truly-unseen subset between the transformer’s best causal cell ($B = 20$, 90.7%) and the Conv1D’s worst ($B = 5$, 52.0%), strengthening the architectural claim from Table 5.2.

Table 5.10: Near-domain CVEfixes external validation: seed-42 trained models, one per configuration, evaluated on the full 212-repository CVEfixes set (2,403 balanced samples). Includes 183 duplicates of training repositories (case-insensitive name matches plus one owner-rename caught by commit-hash overlap) sampled at unseen commits.

Config	T Acc	T F1	T Recall	C Acc	C F1	C Recall
$B = 5$ Symmetric	0.869	0.879	0.951	0.849	0.852	0.866
$B = 10$ Symmetric	0.871	0.881	0.953	0.857	0.864	0.912
$B = 20$ Symmetric	0.884	0.893	0.962	0.848	0.852	0.876
$B = 40$ Symmetric	0.877	0.886	0.955	0.830	0.838	0.878
$B = 5$ Causal	0.879	0.887	0.949	0.591	0.570	0.541
$B = 10$ Causal	0.886	0.894	0.956	0.840	0.850	0.906
$B = 20$ Causal	0.881	0.889	0.952	0.825	0.835	0.878
$B = 40$ Causal	0.876	0.885	0.951	0.831	0.838	0.865

T = transformer, C = Conv1D. Bold marks the highest accuracy or F1 in each temporal mode. The numbers in this table come from the single-seed window sweep on a 2,403-sample balanced test, whereas the 5-seed evaluation in Table 5.4 uses a separate evaluation run on a 2,404-sample balanced test and reports seed 42 at $B = 5$ symmetric as 0.876 rather than 0.869 here. The discrepancy reflects one held-out sample of difference plus a different optimal-threshold choice between the two pipelines, and does not affect any qualitative claim made from either table.

Table 5.11: Truly-unseen window sweep: same 16 seed-42 trained models evaluated on the 29 CVEfixes repositories with no name match (case-insensitive) and no commit-hash overlap against the training set (152 balanced samples).

Config	T Acc	T F1	T Recall	C Acc	C F1	C Recall
$B = 5$ Symmetric	0.868	0.881	0.974	0.829	0.842	0.908
$B = 10$ Symmetric	0.882	0.889	0.947	0.875	0.886	0.974
$B = 20$ Symmetric	0.842	0.854	0.921	0.881	0.883	0.895
$B = 40$ Symmetric	0.854	0.864	0.921	0.848	0.850	0.855
$B = 5$ Causal	0.868	0.882	0.987	0.520	0.490	0.461
$B = 10$ Causal	0.875	0.883	0.947	0.841	0.850	0.895
$B = 20$ Causal	0.907	0.914	0.974	0.862	0.865	0.882
$B = 40$ Causal	0.887	0.897	0.974	0.848	0.852	0.868

The deployment recommendation remains $B = 20$ symmetric. On the near-domain set it ties for the best external accuracy and F1 to within 0.2 percentage points; on the truly-unseen subset it falls to 84.2% / 85.4% (single seed), the largest truly-unseen-versus-near-domain gap of any transformer cell. Two readings of this are consistent with the rest of the data: (i) symmetric pooling with $B = 20$ is the configuration most sensitive to the small-sample

noise of $n = 29$ repositories, and (ii) on truly-unseen data the model favours causal pooling, which reaches 90.7% at $B = 20$ and 88.7% at $B = 40$. The deployment recommendation is retained because $B = 20$ symmetric remains the most stable cross-seed configuration internally (Table 5.3, std 0.018 accuracy) and is within 1 percentage point of the truly-unseen 5-seed mean (0.842 ± 0.016). Across all eight transformer cells, truly-unseen accuracy spans 84% to 91%, while the Conv1D collapses at causal $B = 5$ in both views (59.1% near-domain, 52.0% truly-unseen).

Across the 5-seed evaluation reported in Table 5.4 ($B = 5$ symmetric, all five seeds), the model averages $87.3\% \pm 0.3\%$ accuracy / $93.7\% \pm 0.9\%$ recall on the near-domain set and $84.2\% \pm 1.6\%$ accuracy / $92.6\% \pm 2.4\%$ recall on the truly-unseen subset, confirming that the headline single-seed numbers are not seed-specific outliers. At the natural class imbalance of CVEfixes (1,201 vulnerable among 2,094,039 total commits, ratio 1:1743; one vulnerable commit fewer than the 1,202 in the balanced eval, reflecting a single-sample boundary difference between the two pipelines on the same 212 repositories), the seed-42 transformer at $B = 20$ symmetric reaches ROC-AUC 0.929 and recalls 93.5% of vulnerable commits at threshold 0.5; the Conv1D achieves ROC-AUC 0.917 and 87.9% recall under the same imbalance. All numbers in this subsection depend on having the GraphQL metadata populated: without enrichment, accuracy drops by 4.3 points as several event types are absent from the unenriched CVEfixes timelines.

5.3.8 Comparison to Prior Work

Table 5.12 compares this project’s best results against prior studies. As is standard practice, each study’s best reported result is shown.

Table 5.12: Result comparisons with previous security patch detection studies.

Study	Modalities	Model	Repos	Accuracy	F1
PatchRNN (Wang et al. 2021)	Messages + code diffs	LSTM	313	83.6%	74.7%
SPI (Zhou et al. 2021)	Messages + code diffs	LSTM + CNN	4 (C)	–	87.9%
Sun et al. (2020)	Commit messages	HAN	933	92.8%	92.7%
Zuo et al. (2023)	Commit messages	Transformer	–	91.7%	89.1%
VulCurator (Nguyen et al. 2022)	Messages + issues + code	Conv1D + RoBERTa + CodeBERT	–	–	89.0%
PatchView (Farhi et al. 2025)	Behavioural + messages + code	Conv1D + RoBERTa + CodeBERT	1,679	94.5%	95.1%
Farhi et al. (2023)	Behavioural only	Conv1D	1,389	83.9%	84.1%
Our approach (Best Run)	Behavioural only	Transformer	1,030	88.0%	88.4%
Our approach (CVEfixes)	Behavioural only	Transformer	212*	88.4%	89.3%

* 212 external CVEfixes repositories (full set, near-domain). The truly-unseen subset (29 repositories with no name match and no commit-hash overlap against training) reaches 84.2% / 85.4% at $B = 20$ symmetric (seed 42), with the best causal configuration ($B = 20$) reaching 90.7% / 91.4%.

The studies achieving the highest accuracies all rely on commit message text or source

code, which fail on silent patches or non-target languages respectively. Our approach and Farhi et al. (2023) are the only two studies that detect security patches using exclusively behavioural repository data. Our best single-run configuration (transformer, causal, $B = 20$, seed 42) achieves 88.0% ensemble accuracy and 88.4% F1, exceeding Farhi’s accuracy by 4.1 points and F1 by 4.3 points while using no future context and under a stricter evaluation protocol (GroupKFold).

On external CVEfixes data, the transformer achieves 88.4% accuracy and 89.3% F1 on the 212-repository near-domain set, and 84.2% / 85.4% on the 29-repository truly-unseen subset (seed 42, $B = 20$ symmetric); the best truly-unseen single-seed configuration is causal $B = 20$ at 90.7% / 91.4%. Zuo et al. (2023) report 89.1% F1 using a transformer on commit messages; we achieve 89.3% F1 using behavioural signals alone. VulCurator (Nguyen et al. 2022) reports 89.0% F1 from messages, issues, and code with RoBERTa and CodeBERT; our approach matches that with a single 147,000-parameter model on event metadata. The practical implication is that the behavioural pipeline could serve as a component within multi-modal systems such as PatchView (Farhi et al. 2025), replacing the Conv1D sub-model.

PatchView’s headline 94.5% (Farhi et al. 2025) warrants closer inspection before it is treated as an upper bound. The 94.5% reported in Table 5.12 is PatchView’s full multi-modal system, fusing commit messages through RoBERTa, source code diffs through CodeBERT, and behavioural events through a Conv1D sub-model. Our 88% comes from behavioural events alone, with no text or code modality. A meaningful portion of the headline gap therefore corresponds to the addition of two extra input modalities rather than to a deficit in our model relative to PatchView’s behavioural component. Two further differences also distance the comparison. PatchView partitions by commit rather than by repository, allowing commits from the same repository to appear in both train and test; our GroupKFold protocol rules this out by construction, and the 6 percentage-point drop observed when moving from standard k-fold to GroupKFold during the methodological-correction phase gives a concrete sense of how much evaluation protocol alone can account for. PatchView also includes post-commit events inside its window, which removes the deployment-realism constraint that dominates our causal results. Under symmetric windows our transformer reaches 83.8% to 85.0% across $B = 5$ to $B = 40$, and under causal windows it holds 84.9% to 85.2%, while the Conv1D collapses to 69.4% at causal $B = 5$. Direct cross-study comparison has limitations; each study uses different datasets and protocols, so Table 5.12 gives approximate positioning rather than strict like-for-like comparison.

5.3.9 Error Analysis

Analysis of ensemble predictions on the 548-sample held-out test (seed 42, $B = 20$ symmetric) reproduces the seed-42 confusion shown in Figure 5.2 (TN = 226, FP = 48, FN = 23, TP = 251). The roughly 2:1 ratio of false positives to false negatives is consistent across the study: the model flags 17.5% of benign commits and misses 8.4% of vulnerable ones, so a security team would see roughly one false alarm per five true positives, workable for manual triage. False positives tend to be large benign commits with high `churn_diversity` and `commit_urgency`; false negatives are subtle patches with low code churn. The threshold-

independent discrimination quality is shown in Figure 5.11: the ROC curve reaches $\text{TPR} > 0.8$ at $\text{FPR} \approx 0.15$ ($\text{AUC} = 0.919$) and the precision-recall curve holds precision above 0.8 for recall up to 0.9 ($\text{AP} = 0.891$).

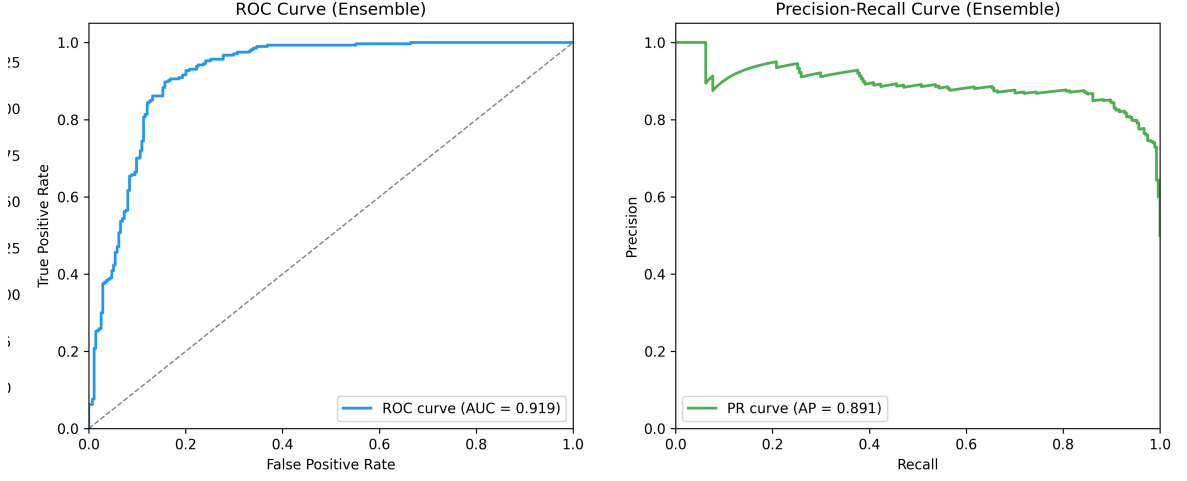


Figure 5.11: ROC and precision-recall curves for the ensemble (seed 42, $B = 20$ symmetric, held-out test set). $\text{AUC} = 0.919$, $\text{AP} = 0.891$.

5.3.10 Attention Pattern Analysis

To examine what the transformer learns, attention weights were extracted from Layer 1 ($B = 20$ symmetric, seed 42) and averaged over 50 correctly classified samples per class. The averaged attention is close to uniform (~ 0.025 per position of 40), expected when aggregating over diverse sequences. The difference map reveals a consistent pattern: for vulnerable commits, the model attends slightly less to the target position and slightly more to events approximately 8 positions before and after. This suggests the model distinguishes patches partly by surrounding temporal context.

Although all heads operate near maximum entropy (indicating subtle redistributions rather than sharp focus), they specialise in the direction of those redistributions. Head 1 suppresses the target position and redistributes attention 8 steps away in both directions. Head 2 shifts attention toward post-commit positions for vulnerable samples, potentially capturing delayed reactions. Head 4 favours pre-commit positions, attending to longer-term history. Head 3 is near-uniform.

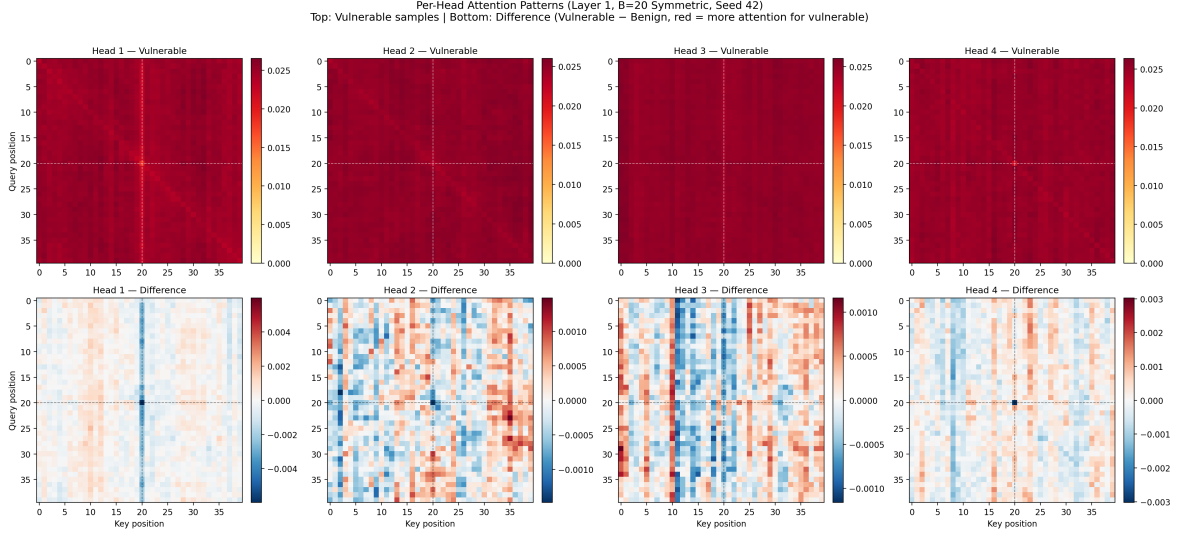


Figure 5.12: Per-head attention patterns from Layer 1 ($B = 20$ symmetric, seed 42). Top row: averaged attention weights for vulnerable samples on a sequential “hot” colormap, where deeper red indicates a higher attention weight from the query position (rows) onto the key position (columns). Bottom row: the difference map (vulnerable – benign) on a diverging red–blue colormap, where red cells mark query–key pairs that receive more attention on vulnerable samples than on benign ones, blue cells mark pairs that receive less, and white cells indicate no class-conditional difference; each colour bar reports the per-head numerical scale.

5.3.11 Summary of Findings and Limitations

RQ1: The transformer achieves statistically significant improvements over Conv1D in accuracy ($p = 0.002$, $d = 1.36$), F1 ($p < 0.001$, $d = 1.68$), and ROC-AUC ($p = 0.002$, $d = 1.39$), with the advantage growing from +0.9 to +2.6 points with window size. Feature engineering contributes more than architecture: removing the 16 temporal features causes a 3.2 point drop, larger than the transformer advantage at any single symmetric window size.

RQ2: The causal transformer achieves 0.849 to 0.852 across all window sizes, matching symmetric performance. The Conv1D collapses to 0.694 at causal $B = 5$. This positions the transformer as viable for real-time deployment.

RQ3: The transformer is largely insensitive to window size, holding 0.838 to 0.850 accuracy across $B = 5$ to $B = 40$. The Conv1D declines steadily from $B = 5$ to $B = 40$, which is what produces the widening transformer–Conv1D gap as window size increases. $B = 20$ offers a practical balance.

Limitations. The training data covers only repositories with publicly disclosed CVEs. Benign sample selection causes 1.8% standard deviation across seeds. The evaluation is retrospective rather than real-time. The model depends on complete event type coverage (4.3 point degradation when absent). The dataset size ($\sim 5,400$) constrains model capacity.

In summary, the transformer improves on the Conv1D baseline across every window configuration with statistically significant effects, maintains accuracy under causal (deployment-realistic) windows where the Conv1D collapses, and generalises to the CVEfixes external set at 88.4% / 96.2% (212-repository near-domain, seed 42, $B = 20$ sym) and 84.2% / 92.1% on the truly-unseen 29-repository subset, with the deployment-relevant causal $B = 20$ configuration reaching 90.7% / 97.4% on that same subset.

Chapter 6

Conclusion and Future Work

6.1 Conclusions

This project set out to evaluate whether a transformer applied to behavioural repository data can improve security patch detection under realistic temporal constraints. The evaluation covered 80 training runs across five random seeds, four window sizes, and two temporal modes, and the results support all three research questions.

On RQ1, the transformer achieves statistically significant improvements over Conv1D (accuracy $p = 0.002$, $d = 1.36$; F1 $p < 0.001$, $d = 1.68$; ROC-AUC $p = 0.002$, $d = 1.39$), confirmed by parametric and non-parametric tests. Feature engineering contributes more than the architecture change alone: removing the 16 derived temporal features causes a 3.2 point accuracy drop, larger than the transformer’s advantage at any single symmetric window size. All nine of the most important features by permutation importance are newly engineered, suggesting that domain-informed feature design remains essential even with more expressive architectures.

On RQ2, the transformer maintains 0.849 to 0.852 accuracy under causal windows, matching or slightly exceeding its symmetric performance. This was the most surprising finding: the transformer shows no measurable degradation from removing future context, implying that post-commit events in symmetric windows carry no additional discriminative signal for this architecture. The Conv1D tells a very different story, collapsing to 0.694 at causal $B = 5$, a 14.7 point drop. This divergence is architecturally explained: the Conv1D’s local kernel and position-destroying max-pooling cannot compensate for lost context, whereas the transformer’s global attention and positional embeddings can. This positions the transformer as viable for real-time deployment.

On RQ3, the transformer is largely insensitive to window size (accuracy stays in the 0.838 to 0.850 range across $B = 5, 10, 20, 40$ symmetric), while the Conv1D declines steadily from 0.841 at $B = 5$ to 0.812 at $B = 40$. The transformer’s growing advantage as B increases is therefore driven by Conv1D’s degradation, not by transformer improvement. $B = 20$ is selected as the practical balance.

Beyond the research questions, external validation on CVEfixes is the strongest result of the project. On the primary configuration (seed 42, symmetric $B = 5$) the transformer reaches 87.6% accuracy and 95.1% recall on the near-domain set (212 repositories, 183

sharing repository identity with training at unseen commits) and 86.8% accuracy and 97.4% recall on the 29 truly-unseen repositories; recall is preserved across the two views ($0.951 \rightarrow 0.974$), so the model continues to flag almost every vulnerability on repositories it has never observed. The cell-level picture is even stronger. The deployment-relevant causal $B = 20$ configuration reaches **90.7% accuracy, 91.4% F1, and 97.4% recall** on the truly-unseen subset, exceeding the same configuration’s near-domain accuracy of 88.1% by 2.6 percentage points and missing only 2 of 76 vulnerable commits. Four of eight B /mode configurations are within one percentage point across the two views; the Conv1D collapse at causal $B = 5$ becomes *more* severe on truly-unseen data (52.0% vs 59.1%), strengthening the case that the architectural divergence is a real generalisation property rather than an artefact of repository identity. Taken together, a behavioural transformer that never reads repository name, commit message, or source code can flag silent security patches at 90%+ accuracy on repositories it has never seen, using only the patterns of when and how developers interact with their repositories. Multi-modal systems like PatchView (Farhi et al. 2025) achieve higher absolute accuracy by combining behavioural, message, and code modalities, but the behavioural pipeline evaluated here is a substantially stronger drop-in component than the convolutional baselines they currently use.

6.2 Future Work

Several directions emerge. Recovering the nine event types still absent from CVEfixes test data via GitHub Archive on BigQuery could close the remaining gap; enrichment already improved accuracy by 4.3 points. The negative sampling strategy warrants investigation, as the 1.8% cross-seed standard deviation is driven by benign commit selection, and difficulty-aware sampling could improve discrimination of borderline cases. Self-supervised pre-training using masked event prediction on unlabelled sequences could overcome the capacity constraints limiting model size to $d_{\text{model}} = 64$. Multi-scale ensembles at different window sizes could capture complementary patterns. Real-time evaluation via GitHub webhook integration would reveal practical considerations around data latency. Finally, incorporating file path patterns or dependency graph changes, without language dependence, could improve detection of subtle low-churn patches.

Chapter 7

Reflection on Learning

7.1 Evaluation Rigour versus Inherited Methodology

The most impactful lesson was discovering the inherited codebase’s evaluation was fundamentally flawed. Replacing standard k-fold with GroupKFold eliminated approximately 6 percentage points of inflated accuracy, revealing that what looked like a well-performing system generalised far worse than reported. This experience taught me that evaluation methodology deserves the same scrutiny as model architecture, and that whenever samples share structural properties such as originating from the same repository, the evaluation protocol must account for those dependencies.

7.2 Debugging Inherited Code

Working with someone else’s codebase required a different skill set. Across the project, 19 issues were identified, ranging from silent data corruption to subtle methodological errors like the causal window excluding the target commit. Several bugs were intermittent and produced no error messages. The most valuable technique I developed was validating intermediate outputs at every pipeline stage, logging shape, range, and distribution of feature matrices after each transformation.

7.3 Feature Engineering versus Architecture Complexity

The architecture exploration programme provided an initially counterintuitive lesson: feature engineering often yields larger improvements than architectural innovation on modestly sized datasets. Increasing transformer capacity degraded ensemble accuracy due to overfitting. Standard regularisation techniques similarly failed. In contrast, six rounds of iterative feature engineering collectively improved ensemble accuracy by 3.4 points, with the two most important features (`churn_diversity` and `commit_urgency`) designed through domain reasoning. For limited-data problems where capacity is not the binding constraint, I would now prioritise feature engineering before architectural alternatives.

7.4 Data Alignment and External Validation

Constructing the CVEfixes set taught me an unexpected lesson about data alignment. An initial 3 to 4 point gap could plausibly have been interpreted as overfitting, but investigation revealed it was caused by event type misalignment. Enriching the dataset to recover five additional event types from cached GraphQL data plus an inferred PushEvent type improved external accuracy by 4.3 points and closed the gap. Without this investigation, the project narrative would have incorrectly concluded that the model partially overfits to its training distribution.

7.5 Time Management and Experimental Planning

The experimental programme (80 main training runs of 10 fold models each, plus 23 architecture exploration experiments) required careful planning on consumer hardware. Early ad-hoc experiments wasted computation. Shifting to a structured, hypothesis-grouped approach substantially improved efficiency. In retrospect, establishing pre-defined stopping criteria for each research question at the outset would have freed up more time for the write-up phase.

7.6 Reproducibility as Engineering Discipline

Ensuring reproducibility proved to be as much a productivity tool as a scientific principle. The discovery that `os.listdir()` returns files in non-deterministic order, causing 2–3% metric variance from a seemingly innocuous system call, illustrated that reproducibility failures can arise from system-level behaviours invisible at the application level. Achieving full determinism required controlling five separate randomness sources and enabling TensorFlow’s deterministic operations mode. In future projects, I intend to establish seed control and result caching infrastructure from the outset.

7.7 Use of AI Assistance

Throughout the project I used a large language model assistant (Claude) as a development collaborator, principally for tasks where the cost of context-switching outweighed the difficulty of the underlying decision. The assistant was most useful in three areas.

The first was debugging. When the inherited codebase produced silent failures, I pasted stack traces, intermediate tensor shapes, and short code excerpts and asked for hypotheses. The assistant rarely produced the correct root cause on the first attempt, but its suggestions narrowed the search space and occasionally surfaced angles I had not considered, such as checking whether file ordering on disk was deterministic. The discipline of articulating the problem clearly enough for an external collaborator was itself valuable; the act of phrasing the bug often clarified it before any reply was returned.

The second was orchestration. The 80-run sweep and the 16-cell CVEfixes window sweep both ran overnight on the laptop, and writing the bash scaffolding around them (timestamping, log capture via `tee`, stale-output detection, `grep` patterns to extract the per-fold CRITICAL

lines later consumed by `compute_paired_stats.py`) was tedious rather than conceptually difficult. I drafted the structure myself and asked the assistant to fill in the syntactically fiddly parts. Letting the runs proceed unattended overnight, then auditing the captured logs in the morning, recovered roughly a day per week of effective working time.

The third was idea generation. When stuck on which derived feature to engineer next, short conversations produced hypotheses I could then test empirically; the eventual high-importance features (`churn_diversity`, `commit_urgency`) were arrived at through domain reasoning rather than direct suggestion, but several discarded candidates would not have been considered without the prompt. Throughout, I treated assistant output as a starting point rather than a result, verifying every produced claim against the code or the literature before incorporating it. The clearest lesson is that AI assistance accelerates the parts of research that are bookkeeping but does not replace the parts that require judgement.

References

- Ba, J.L., Kiros, J.R. and Hinton, G.E. 2016. Layer Normalization [Preprint]. Available at: <https://arxiv.org/abs/1607.06450> [Accessed: 26 April 2026].
- Bhandari, G., Naseer, A. and Moonen, L. 2021. CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software [Preprint]. Available at: <https://arxiv.org/abs/2107.08760> [Accessed: 13 April 2026].
- Farhi, N., Koenigstein, N. and Shavitt, Y. 2023. Detecting Security Patches via Behavioural Data in Code Repositories [Preprint]. Available at: <https://arxiv.org/abs/2302.02112> [Accessed: 6 February 2026].
- 2025. PatchView: Multi-modality detection of security patches. *Computers and Security*. Available at: <https://dl.acm.org/doi/10.1016/j.cose.2025.104356> [Accessed: 13 April 2026].
- Goodin, D. 2024. What we know about the xz Utils backdoor that almost infected the world. Available at: <https://arstechnica.com/security/2024/04/what-we-know-about-the-xz-utils-backdoor-that-almost-infected-the-world/> [Accessed: 13 April 2026].
- Hiesgen, R., Nawrocki, M., Schmidt, T.C. and Wählisch, M. 2024. The Log4j Incident: A Comprehensive Measurement Study of a Critical Vulnerability. Available at: https://www.researchgate.net/publication/382952425_The_Log4j_Incident_A_Comprehensive_Measurement_Study_of_a_Critical_Vulnerability [Accessed: 13 April 2026].
- Khalil, M. 2025. Vulnerability Statistics 2025: Record CVEs, Zero-Days and Exploits. Available at: <https://deepstrike.io/blog/vulnerability-statistics-2025> [Accessed: 6 February 2026].
- Loshchilov, I. and Hutter, F. 2019. Decoupled Weight Decay Regularization. *International Conference on Learning Representations (ICLR)*. Available at: <https://arxiv.org/abs/1711.05101> [Accessed: 26 April 2026].
- Nguyen, T.G., Le-Cong, T., Kang, H.J., Le, X.-B.D. and Lo, D. 2022. VulCurator: A Vulnerability-Fixing Commit Detector [Preprint]. Available at: <https://arxiv.org/abs/2209.03260> [Accessed: 13 April 2026].
- Reuters 2010. Security firm reveals Microsoft’s ‘silent’ patches. Available at: <https://www.reuters.com/article/world/security-firm-reveals-microsoft-s-silent-patches-idUS1763837303/> [Accessed: 7 February 2026].
- Shostack, A. 2014. Threat Modeling: Designing for Security. Wiley. Available at: https://librarysearch.cardiff.ac.uk/permalink/44WHELF_CAR/1r4ug5q/alma9911638983102420 [Accessed: 13 April 2026].

- Stefanoni, A., Girdzijauskas, S., Jenkins, C., Kefato, Z., Sbattella, L., Scotti, V. and Wareus, E. 2022. Detecting Security Patches in Java Projects Using NLP Technology. *ICNLSP*. Available at: <https://aclanthology.org/2022.icnls-1.6> [Accessed: 7 February 2026].
- Sun, M., Wang, W., Feng, H., Sun, H. and Zhang, Y. 2020. Identify Vulnerability Fix Commits Automatically Using Hierarchical Attention Network. Available at: https://www.researchgate.net/publication/341352657_Identify_Vulnerability_Fix_Commits_Automatically_Using_Hierarchical_Attention_Network [Accessed: 13 April 2026].
- Vaswani, A. et al. 2017. Attention is All You Need. *Advances in Neural Information Processing Systems*. Available at: <https://arxiv.org/abs/1706.03762> [Accessed: 16 February 2026].
- Verizon 2025. *2025 Data Breach Investigations Report*. Tech. rep. Verizon. Available at: <https://www.verizon.com/business/resources/reports/dbir/> [Accessed: 10 February 2026].
- Wang, X., Wang, S., Feng, P., Sun, K., Jajodia, S., Benchaaboun, S. and Geck, F. 2021. PatchRNN: A Deep Learning-Based System for Security Patch Identification. *MILCOM*. Available at: <https://arxiv.org/abs/2108.03358> [Accessed: 15 February 2026].
- Xiong, R. et al. 2020. On Layer Normalization in the Transformer Architecture. *ICML*. Available at: <https://arxiv.org/abs/2002.04745> [Accessed: 13 April 2026].
- Zhang, H., Cissé, M., Dauphin, Y.N. and Lopez-Paz, D. 2018. mixup: Beyond Empirical Risk Minimization. *International Conference on Learning Representations (ICLR)*. Available at: <https://arxiv.org/abs/1710.09412> [Accessed: 26 April 2026].
- Zhou, Y., Siow, J.K., Wang, C., Liu, S. and Liu, Y. 2021. SPI: Automated Identification of Security Patches via Commits [Preprint]. Available at: <https://arxiv.org/abs/2105.14565> [Accessed: 12 February 2026].
- Zuo, F., Zhang, X., Song, Y., Rhee, J. and Fu, J. 2023. Commit Message Can Help: Security Patch Detection in Open Source Software via Transformer. *IEEE*. Available at: <https://ieeexplore.ieee.org/document/10197730> [Accessed: 7 February 2026].